



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

پایان نامه کارشناسی
مهندسی کامپیوتر

مدیریت قابلیت اطمینان و انرژی مبتنی بر یادگیری ماشین در رایانش لبه خودرویی

نگارش

پارسا شهیدی

استاد راهنما

دکتر محسن انصاری

مرداد ۱۴۰۵

سپاس

از استاد بزرگووارم که با کمک‌ها و راهنمایی‌های بی‌دریغشان، مرا در به سرانجام رساندن این پایان‌نامه یاری داده‌اند، تشکر و قدردانی می‌کنم. همچنین از همکاران عزیزی که با راهنمایی‌های خود در بهبود نگارش این نوشتار سهیم بوده‌اند، صمیمانه سپاسگزارم.

چکیده

با توجه به رشد روزافزون خودروهای خودران و اهمیت کاهش مصرف انرژی و افزایش ایمنی در این سامانه‌ها، این پروژه به دنبال ارائه‌ی روشی نوآورانه است که در کنار مدیریت مصرف انرژی خودروها، قابلیت اطمینان را نیز مدیریت کرده و خللی در ایمنی خودروها وارد نشود. روش‌های مدیریت انرژی باعث کاهش قابلیت اطمینان شده و از طرفی کاهش قابلیت اطمینان در این سامانه‌ها ایمنی را کاهش می‌دهد. در سال‌های اخیر، با افزایش دستگاه‌های اینترنت اشیا اعم از خودروهای خودران هوشمند و پدیدار شدن چالش‌های جدید در رایانش ابری و نیاز به پردازش بی‌درنگ در این دستگاه‌ها، رایانش لبه به عنوان یک رویکرد مناسب برای رفع این چالش‌ها ظهور کرده است. محاسبات لبه یکی از فناوری‌های نوظهور در حوزه رایانش ابری است که با کاهش تأخیر و افزایش بهره‌وری، عملکرد بهتری نسبت به پردازش ابری ارائه می‌دهد. این پروژه با هدف بهبود هم‌زمان مدیریت انرژی و قابلیت اطمینان در خودروهای خودران هوشمند، با بهره‌گیری از الگوریتم‌های یادگیری ماشین و رایانش لبه ارائه می‌شود. در این پژوهش، بستر یکپارچه‌ی دوقلوی دیجیتال شامل شبیه‌سازی خودرو، ترافیک و شبکه‌ی خودرو با همه‌چیز به‌همراه پشته‌ی خودرانی مبتنی بر ROS 2 پیاده‌سازی شد و سیاست‌های بدون بارسپاری، بارسپاری تصادفی و بارسپاری مبتنی بر یادگیری تقویتی عمیق به‌صورت تجربی مقایسه گردیدند. نتایج نشان می‌دهد بدون بارسپاری معمولاً کم‌تأخیرتر ولی پرمصرف‌تر است، بارسپاری تصادفی انرژی را کاهش می‌دهد اما تأخیر را افزایش می‌دهد، و سیاست یادگیری تقویتی عمیق باید بر اساس مصالحه‌ی تأخیر-انرژی هر وظیفه قضاوت شود. در کمپین تزریق خطا با 100 آزمایش، 95٪ خطاها توسط رأی‌گیر ماسک شدند، 5٪ به مسیر ایمن حداقلی رفتند و هیچ مورد تشخیص‌نشده‌ای گزارش نشد.

کلیدواژه‌ها: رایانش لبه، رایانش بی‌درنگ، خودرو خودران هوشمند، یادگیری ماشین، قابلیت اطمینان

فهرست مطالب

1	مقدمه	1
1-1	تعریف مسئله	1
2-1	اهمیت موضوع	3
3-1	ادبیات موضوع	3
4-1	اهداف پژوهش	6
5-1	ساختار پایان نامه	7
2	مفاهیم اولیه	8
1-2	دوقلوی دیجیتال و ارزیابی حلقه بسته	8
2-2	پشته‌ی نرم‌افزاری خودروهای خودران	8
1-2-2	قطعه‌بندی معنایی	10
2-2-2	تخمین عمق	10
3-2-2	تشخیص چراغ راهنمایی	10
4-2-2	نقشه‌برداری LiDAR SLAM	12
5-2-2	مکان‌یابی GNSS/IMU	14
6-2-2	برنامه‌ریزی مسیر	14
7-2-2	برنامه‌ریز محلی	14
8-2-2	پایشگر برخورد	16

17	3-2	رایانش لبه و بارسپاری وظایف
19	4-2	شبکه‌ی V2X و اثر آن بر تصمیم بارسپاری
20	5-2	یادگیری تقویتی عمیق برای تصمیم بارسپاری
21	6-2	قابلیت اطمینان و تحمل‌پذیری اشکال
23	7-2	الزامات بی‌درنگ سخت و نرم

3 کارهای پیشین 24

24	1-3	انتخاب سرور و بارسپاری مبتنی بر یادگیری تقویتی
25	2-3	قابلیت اطمینان و زمان‌بندی استنتاج عمیق در لبه
25	3-3	بارسپاری انرژی‌آگاه و چندخدمتی
26	4-3	مصالحه‌ی کیفیت-تأخیر در سناریوهای V2X
27	5-3	جمع‌بندی و جایگاه پژوهش حاضر

4 پیاده‌سازی 29

29	1-4	بستر اجرا و ابزارها
31	2-4	پیاده‌سازی وظایف
32	3-4	مسیر بارسپاری وظایف
33	4-4	پیاده‌سازی تحمل‌پذیری اشکال
33	5-4	خط لوله‌ی ارزیابی
33	6-4	چالش‌های پیاده‌سازی
35	7-4	جمع‌بندی فصل

5 ارزیابی و تحلیل نتایج 36

36	1-5	تنظیم آزمایش
36	2-5	زمان اجرا و انرژی
37	3-5	قابلیت اطمینان

38 4-5 جمع بندی نتایج

40 نتیجه گیری 6

40 1-6 جمع بندی

41 2-6 محدودیت ها

41 3-6 پیشنهاد کارهای آتی

43 مراجع

48 واژه نامه

فهرست جدول‌ها

- 1-2 وظایف پیاده‌سازی شده در پشته‌ی خودرانی پروژه 11
- 1-3 مقایسه‌ی کارهای پیشین مرتبط با بار سپاری لبه و جایگاه پژوهش حاضر 28
- 1-4 API های نوشته شده برای اتصال شبیه‌سازها و پروتکل آن‌ها 31
- 2-4 بازه‌ی نمونه برداری یکنواخت پارامترهای وظایف مصنوعی و ادراکی 31

فهرست شکل‌ها

- 1-1 معماری عامل PPO در روش EPTask برای زمان‌بندی انرژی‌آگاه و اولویت‌آگاه در
لبه‌ی خودرویی پویا [1]. 4
- 2-1 شماتیک شبیه‌ساز شبکه‌ی MEC و راهبرد تخصیص وظایف RASOUL همراه با
خروجی‌های نسبت موفقیت، مصرف انرژی و کیفیت سرویس [2]. 6
- 1-2 معماری پشته‌ی نرم‌افزاری خودرانی پروژه. فلش‌های نقطه‌چین ارتباطاتی هستند که قطع
شده‌اند تا خروجی وظایف نرم روی وظایف سخت اثر نگذارد و این‌گونه نرم محسوب
شوند. 9
- 2-2 شماتیک قید بستن حلقه در گراف فاکتورها: پس از بازدید مجدد یک ویژگی محیطی،
قید نسبی بین گره‌ی جاری و یک گره‌ی قبلی افزوده می‌شود تا رانش تجمعی مسیر
کاهش یابد. 14
- 3-2 هندسه‌ی کنترلر Stanley: خطای عرضی e از محور جلو تا مسیر، خطای سمت (در
شکل ψ و در متن ψ_e)، زاویه‌ی فرمان δ ، فاصله‌ی محورها L و سرعت محور جلو v_f [3]. 16
- 4-2 مفهوم بارسپاری محاسباتی در رایانش لبه‌ی خودرویی. 17
- 5-2 تأثیر بار و اندازه‌ی وظایف بر تأخیر میانگین شبکه: (چپ) افزایش تأخیر با تعداد وظایف
و (راست) افزایش تأخیر با اندازه‌ی واقعی وظایف ادراکی. 19
- 6-2 سازوکار تحمل‌پذیری اشکال مبتنی بر TMR: اجرای سه‌گانه‌ی وظیفه‌ی سخت، رأی‌گیر
برای حذف خروجی‌های دیررس/غیرقابل قبول، پایشگر تنزل و مسیر ایمن حداقلی. 22
- 1-3 معماری عامل DDQN برای تصمیم مشترک بارسپاری و تخصیص منابع در MEC
انرژی‌کارا: شبکه‌ی اصلی/هدف، بافر بازپخش تجربه و به‌روزرسانی با گرادینان زیان [4]. 27

- 1-4 معماری هم‌شبیه‌سازی: اتصال SUMO، CARLA، Veins/OMNeT++ و پشت‌هی
 2 ROS. 30
- 2-4 پیاده‌سازی مسیر بارسپاری: تصمیم‌گیرنده، اجرای محلی، پل شبکه و نسخه‌ی دوردست. 32
- 3-4 پیاده‌سازی تحمل‌پذیر اشکال برای سامانه‌ی اجتناب از برخورد: سه نسخه‌ی CAS،
 رأی‌گیر TMR، دروازه‌ی داوری و مسیر fail-safe. 34
- 1-5 مقایسه‌ی زمان اجرای نرمال‌شده و انرژی نرمال‌شده برای سیاست‌های بدون بارسپاری،
 تصادفی و یادگیری تقویتی عمیق روی وظایف ادراکی و مصنوعی. 37
- 2-5 خلاصه‌ی ماسک‌شدن خطا و خروجی سامانه در 100 آزمایش تزریق خطا. 38
- 3-5 توزیع تأخیر تشخیص خطا (چپ) و زمان بازیابی (راست). 38

فصل 1

مقدمه

در این فصل، مسئله‌ی مدیریت هم‌زمان بارسپاری¹ وظایف ادراکی و تحمل‌پذیری اشکال² برای وظایف سخت بی‌درنگ³ در پشت‌پشته‌ی نرم‌افزاری خودروهای خودران مطرح می‌شود.

1-1 تعریف مسئله

خودروهای خودران سامانه‌های سایبر-فیزیکی هستند که از طریق حسگری⁴، پردازش و عملگر⁵، با محیط فیزیکی تعامل دارند و باید قیود سخت ایمنی و زمان‌بندی را رعایت کنند. پشت‌پشته‌ی نرم‌افزاری خودرو یک خط لوله‌ی⁶ پایان‌به‌پایان⁷ را به‌صورت مجموعه‌ای از وظایف هم‌زمان اجرا می‌کند. اجرای هم‌زمان این خط لوله روی رایانه‌ی توکار⁸ با محدودیت توان، حرارت و ظرفیت باتری، به‌خودی‌خود دشوار است؛ تأمین ضمانت کارکردی و کارایی سخت بی‌درنگ در حضور خطا دشوارتر است، زیرا ادراک⁹ پرمصرف و کنترل ایمنی محور باید روی همان منابع محدود هم‌زیستی داشته باشند.

در چنین بستری دو راه‌حل اصلی پدید می‌آید که جهت‌گیری‌های متفاوتی دارند و مسئله‌ی این پژوهش را شکل می‌دهند.

¹offloading

²fault tolerance

³hard real-time

⁴sensing

⁵actuation

⁶pipeline

⁷end-to-end

⁸embedded

⁹perception

بارسپاری تحت عدم قطعیت شبکه. اجرای کامل خط لوله‌ی ادراک به صورت محلی، بودجه‌ی توان، حرارت، باتری و زمان بندی پذیری را تحت فشار قرار می دهد. رایانش لبه‌ی متحرک و خودرویی با ارسال وظایف سنگین روی پیوند V2X¹⁰ به سرورهای نزدیک می تواند این بار را کم کند؛ اما تخصیص باید میان انرژی، تأخیر¹¹، کیفیت سرویس¹² و نرخ موفقیت وظیفه، در شرایط متغیر بی سیم و سرور، تعادل برقرار کند. با افزایش چگالی ترافیک، محیط مشترک رادیویی شلوغ می شود، تأخیر مسیر دور افزایش می یابد و بارسپاری یک وظیفه‌ی ادراکی حساس به تأخیر، به جای کمک، می تواند مهلت زمانی¹³ آن را نقض کند. بنابراین سیاست¹⁴ بارسپاری باید نسبت به وضعیت شبکه و ترافیک آگاه باشد و انتخاب کند کدام وظایف بارسپاری شوند.

تحمل پذیری اشکال برای وظایف سخت بی درنگ. از سوی دیگر، خودرو خودران ایمنی محور است: با پیچیده تر شدن سامانه، احتمال خطاهای گذرا و ماندگار افزایش می یابد و تکنیک های تحمل پذیری اشکال برای حفظ اجرای صحیح وظایف ضروری می شوند. بهینه سازی هایی که صرفاً بر کاهش انرژی یا بارسپاری تمرکز دارند و قابلیت اطمینان را نادیده می گیرند، می توانند نرخ خطا را بالا ببرند یا مسیر کنترل را بی حفاظ بگذارند. وظایف سخت بی درنگ مانند پایش برخورد، کنترل طولی¹⁵ و مکان یابی باید حتی در شرایط خطا دار نیز مهلت خود را برآورده کنند و خروجی قابل استفاده تولید نمایند؛ از کار افتادن بی اعلان مسیر اصلی نباید خودرو را از کار بیندازد. تحمل پذیری اشکال قابلیت اطمینان¹⁶ را باز می گرداند، اما سربار زمانی و منابعی دارد که باید در کنار قیود بی درنگ لحاظ شود.

کارهای پیشین معمولاً این دو راه حل را جداگانه بررسی کرده اند: مطالعات بارسپاری مبتنی بر یادگیری در رایانش لبه، مصالحه‌ی¹⁷ تأخیر-انرژی-موفقیت را روی مدل های انتزاعی شبکه بهبود می دهند؛ و روش های تعبیه شده‌ی آگاه به قابلیت اطمینان، توان و تحمل پذیری اشکال را زیر مهلت زمانی هم مدیریت می کنند. آنچه کمتر دیده می شود، یک ارزیابی حلقه بسته¹⁸ در خودرو خودران است که در آن (۱) تصمیم بارسپاری ادراک تحت تأخیر واقعی و هم بسته با ترافیک V2X گرفته شود و (۲) وظایف سخت بی درنگ روی خودرو پشت یک مسیر تحمل پذیر اشکال باقی بمانند، و اثر هر دو در یک اجرای واحد با معیارهایی چون کیفیت سرویس مهلت نرم، موفقیت سرویس، زمان بندی پذیری، انرژی و تأخیر بازیابی اندازه گیری شود.

¹⁰Vehicle-to-Everything

¹¹latency

¹²Quality of Service (QoS)

¹³deadline

¹⁴policy

¹⁵longitudinal control

¹⁶reliability

¹⁷trade-off

¹⁸Closed-loop

مسئله‌ی این پایان‌نامه دقیقاً همین شکاف را هدف قرار می‌دهد: طراحی و ارزیابی یک پشته‌ی خودرانی¹⁹ حلقه‌بسته که با وزن برابر، بارسپاری آگاه از شبکه را برای وظایف ادراکی قابل بارسپاری و تحمل‌پذیری اشکال را برای وظایف سخت بی‌درنگ، روی یک بستر هم‌شبه‌سازی با ارزیابی شبکه در حلقه بررسی کند؛ به گونه‌ای که وظایف ایمنی‌محور حتی در حضور خطا همچنان در مهلت زمانی خود قابل دسترس بمانند و بار ادراک نیز تحت تأخیر واقع گرایانه‌ی شبکه مدیریت شود.

2-1 اهمیت موضوع

خودروهای خودران یکی از مهم‌ترین فناوری‌های حمل‌ونقل آینده به‌شمار می‌روند و می‌توانند ایمنی جاده، کارایی ترافیک و تجربه‌ی جابه‌جایی را به‌طور چشمگیری بهبود دهند. با این حال، عملیاتی شدن آن‌ها وابسته به پشته‌ی نرم‌افزاری سنگینی است که روی منابع محدود خودرو اجرا می‌شود؛ بنابراین هر پیشرفتی که مصرف انرژی و قابلیت اطمینان این پشته را هم‌زمان بهتر کند، برای گسترش واقعی این فناوری اهمیت دارد.

این پژوهش دقیقاً در همین راستا است و به چالش مصرف باتری خودرو کمک می‌کند: با بارسپاری آگاه از شبکه برای وظایف ادراکی پرمصرف، بخشی از بار محاسباتی از رایانه‌ی توکار برداشته می‌شود و مصرف انرژی سطح‌وظیفه کاهش می‌یابد، بدون آن‌که تصمیم بارسپاری نسبت به وضعیت ترافیک و تأخیر V2X کور باشد.

از سوی دیگر، اهمیت موضوع تنها در صرفه‌جویی انرژی نیست. در سامانه‌ای ایمنی‌محور، بروز خطا داخل خودرو نباید به فاجعه تبدیل شود؛ حتی یک از کار افتادن بی‌اعلان در مسیر کنترل می‌تواند فرمان سرعت را قطع کند و ایمنی سرنشین را به خطر بیندازد. از این رو، تحمل‌پذیری اشکال و حفظ دسترس‌پذیری وظایف سخت بی‌درنگ در کنار مدیریت انرژی ضروری است تا خرابی جزئی به حادثه منجر نشود.

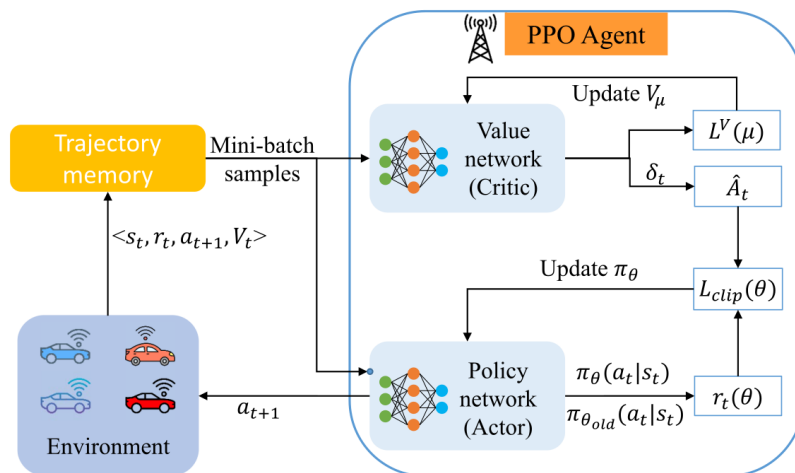
3-1 ادبیات موضوع

ادبیات مرتبط را می‌توان در چند محور خلاصه کرد؛ مرور تفصیلی‌تر در فصل سوم آمده است.

بارسپاری مبتنی بر یادگیری در رایانش لبه‌ی خودرویی²⁰. روش‌هایی مانند DRPL و TODM تصمیم بارسپاری و تخصیص منابع را با یادگیری تقویتی عمیق برای تشخیص شیء یا معماری ابر-لبه-خودرو

¹⁹autonomous driving stack

²⁰Vehicular Edge Computing (VEC)



شکل 1-1: معماری عامل PPO در روش EPTask برای زمان‌بندی انرژی‌آگاه و اولویت‌آگاه در لبه‌ی خودرویی پویا [1].

بهینه‌سازی می‌کنند و معمولاً سودمندی زمانی یا هزینه‌ی تأخیر-انرژی را روی مدل‌های انتزاعی گزارش می‌دهند [5, 6]. لی و همکاران نیز با الگوریتم EPTask مبتنی بر PPO چندعملی، زمان‌بندی و تخصیص منابع را در لبه‌ی خودرویی پویا با در نظر گرفتن اولویت وظیفه و مصرف انرژی به صورت مشترک حل کرده‌اند [1]. شکل 1-1 معماری عامل PPO در آن کار را نشان می‌دهد: محیط ترافیکی، حافظه‌ی مسیر و شبکه‌های ارزش/سیاست با هدف بریده‌شده برای به‌روزرسانی پایدار سیاست.

آنسره و همکاران تخصیص بهینه‌ی منابع محاسباتی را در سامانه‌های لبه‌ی اینترنت اشیاء با DRL دوگانه و بهینه‌سازی لیاپانوف برای بیشینه‌کردن بهره‌وری انرژی بلندمدت پیشنهاد کرده‌اند [7]. این آثار نشان می‌دهند یادگیری تقویتی برای تصمیم MEC/VEC مؤثر است، اما کمتر تأخیر V2X هم‌بسته با ترافیک را به خط لوله‌ی ادراک-کنترل زنده تزریق می‌کنند و زمان‌بندی‌پذیری یا بازیابی مسیر کنترل را در همان اجرا نمی‌سنجند [8].

زمان‌بندی و تخصیص منابع انرژی‌آگاه. حسین و همکاران زمان‌بندی وظایف بی‌درنگ روی سامانه‌های لبه‌ی با عملکرد بالا را با ترکیب الگوریتم ژنتیک و انتخاب کمینه‌ی سرعت شدنی (GA-FiFeS) پیشنهاد کرده‌اند تا ضمن رعایت مهلت، مصرف انرژی کاهش یابد [9]. هوا و همکاران تخصیص منابع انرژی‌آگاه را در معماری ناهمگون لبه-ابر با در نظر گرفتن تحرک کاربر مدل کرده و یک الگوریتم مکاشفه‌ای کم‌پیچیدگی ارائه داده‌اند [10]. شو و همکاران تخصیص زیرباند و توان ارسالی را برای ارتباطات V2X با یادگیری تقویتی چندعاملی و با هدف بهره‌وری انرژی تحت قیود کیفیت سرویس بررسی کرده‌اند [11]. این خط پژوهشی انرژی و قیود بی‌درنگ/مخابراتی را جدی می‌گیرد، اما معمولاً از پشته‌ی خودرانی حلقه‌بسته و سنجش هم‌زمان ایمنی مسیر کنترل فاصله دارد.

تحمل‌پذیری اشکال در بارسپاری و افزونگی. لانگ و همکاران بارسپاری خدمت سیار تحمل‌پذیر اشکال را در MEC با تصمیم‌گیری مبتنی بر یادگیری تقویتی پیشنهاد کرده‌اند تا اثر خطاهای سخت‌افزار نرم‌افزار و عدم‌توازن منابع بر کیفیت خدمت کاهش یابد [12]. جیانگ و همکاران تکرار وظیفه در رایانش ابری خودرویی با مهلت سخت را به‌صورت MDP مدل کرده و سیاست بهینه‌ی تخصیص متوازن (BETA) را برای کمینه‌کردن احتمال نقض مهلت ارائه داده‌اند [13]. چوی و همکاران نیز معماری سخت‌افزاری شش‌پیمانه‌ای²¹ مبتنی بر TMR تقریبی (A-TMR) برای پردازش داده‌ی حسگر خودرویی پیشنهاد کرده‌اند که با رأی‌گیری تقریبی و جایگزینی برخط پیمانه‌ی²² معیوب، قابلیت اطمینان را نسبت به TMR کلاسیک افزایش می‌دهد [14]. با این حال این آثار یا در سطح بارسپاری انتزاعی/MEC می‌مانند، یا تکرار وظیفه را در ابر خودرویی بدون پشت‌پشتی ادراک-کنترل می‌بینند، یا افزونگی را در سخت‌افزار حسگر پیاده می‌کنند؛ ترکیب تحمل‌پذیری اشکال نرم‌افزاری مسیر کنترل سخت بی‌درنگ با بارسپاری آگاه از شبکه‌ی ترافیک محور کمتر دیده می‌شود.

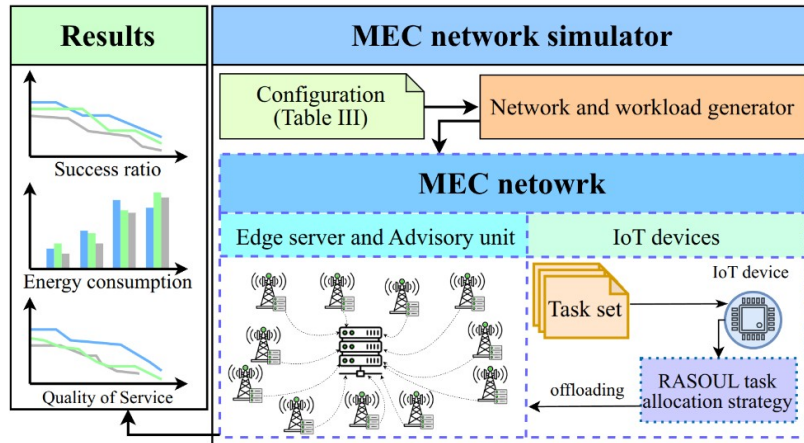
بارسپاری وظایف با کمک V2X. کارهایی مانند بارسپاری تشخیص شیء مبتنی بر YOLO روی لبه/ابر، مصالحه‌ی کیفیت-تأخیر را زیر فشرده‌سازی و شبیه‌سازی شبکه‌ی نسل پنجم بررسی می‌کنند [15, 16, 17]. تمرکز آن‌ها بیشتر بر اندازه‌ی آشکارساز و کدگذاری انتقال است تا سیاست تطبیقی مبتنی بر چگالی ترافیک، و راه‌اندازی شبکه غالباً ایستا است نه رانده‌شده از ترافیک زنده‌ی اطراف.

معیارهای کیفیت سرویس و قابلیت اطمینان، و هم‌شبیه‌سازی. مدل‌هایی مانند DRaM و RASOUL کیفیت سرویس مهلت‌نرم و موفقیت سرویس را برای تخصیص وظایف آگاه‌به‌قابلیت‌اطمینان تعریف می‌کنند؛ در این پایان‌نامه این تعاریف برای ارزیابی سطح وظیفه به کار گرفته می‌شوند [2, 18]. رسولی و همکاران در RASOUL راهبرد تخصیص وظایف آگاه از قابلیت اطمینان را در شبیه‌ساز شبکه‌ی MEC ارزیابی کرده‌اند [2]؛ شکل 1-2 نمای این شبیه‌ساز، شامل تولید بار، واحد مشاوره سرور لبه، دستگاه‌های IoT و معیارهای نسبت موفقیت، انرژی و کیفیت سرویس را نشان می‌دهد.

از سوی دیگر، شبیه‌سازهای حسگری خودرویی خودی، ترافیک خرد و شبکه‌ی V2X معمولاً به‌صورت جداگانه به کار می‌روند و بسترهایی نیز آن‌ها را برای پژوهش‌های تحرک متصل یکپارچه می‌کنند [19, 20, 21, 22, 23]. پژوهش حاضر نیز این سه دامنه را زیر ROS 2 هم‌شبیه‌سازی می‌کند، اما پرسش اصلی‌اش متفاوت است: اثر تصمیم بارسپاری (یادگیری تقویتی یا تصادفی) تحت تأخیر شبکه در حلقه بر کنترل ایمنی، زمان‌بندی‌پذیری، انرژی و بازیابی نگهبان در یک اجرای حلقه‌بسته چیست [24].

²¹modular

²²module



شکل 1-2: شماتیک شبیه‌ساز شبکه‌ی MEC و راهبرد تخصیص وظایف RASOUL همراه با خروجی‌های نسبت موفقیت، مصرف انرژی و کیفیت سرویس [2].

4-1 اهداف پژوهش

هدف کلی این پژوهش، طراحی، پیاده‌سازی و ارزیابی یک پشته‌ی خودرانی حلقه‌بسته است که مدیریت انرژی و قابلیت اطمینان را به‌صورت هم‌زمان، با بارسپاری آگاه از شبکه و تحمل‌پذیری اشکال برای وظایف سخت بی‌درنگ، محقق کند. اهداف جزئی عبارتند از:

- ساخت بستر دوقلوی دیجیتال²³ یکپارچه برای یک خودروی خودی در ترافیک، با صدور وضعیت ترافیک زنده و سرویس تأخیر V2X مبتنی بر مدل شبکه‌ی هم‌بسته با ترافیک.
- شبیه‌سازی و پیاده‌سازی پشته‌ی خودرانی به‌منظور تعریف و ارزیابی وظایف واقع‌گرایانه در محیط شبیه‌سازی.
- طراحی سیاست بارسپاری مبتنی بر یادگیری تقویتی عمیق آگاه از فضای حالت (سطح ازدحام هر ناحیه، موقعیت ناحیه‌ای خودروی خودی و سطح احتمال بارسپاری هر ناحیه) و مقایسه‌ی آن با خطوط‌پایه‌ی تصادفی و بدون بارسپاری.
- پیاده‌سازی سازوکار TMR²⁴ [25] برای افزایش تحمل‌پذیری اشکال و تضمین اجرای پایدار وظایف سخت بی‌درنگ، به‌همراه استخراج نتایج و مقایسه‌ی عملکرد با حالت بدون تحمل‌پذیری اشکال.

²³Digital Twin

²⁴Triple Modular Redundancy

5-1 ساختار پایان نامه

این پایان نامه در شش فصل به شرح زیر ارائه می شود. فصل دوم مفاهیم اولیه‌ی مورد نیاز شامل رایانش لبه‌ی خودرویی، بارسپاری وظیفه، یادگیری تقویتی، قیود بی درنگ سخت و تحمل پذیری اشکال را معرفی می کند. فصل سوم به مرور و نقد کارهای پیشین در انتخاب سرور و بارسپاری مبتنی بر یادگیری، زمان بندی قابلیت اطمینان آگاه استنتاج عمیق، و بارسپاری انرژی آگاه می پردازد. فصل چهارم جزئیات پیاده سازی سامانه، ابزارها، مسیر بارسپاری، زیرسامانه‌ی تحمل پذیر اشکال و چالش های عملی پیاده سازی را شرح می دهد. فصل پنجم نتایج ارزیابی تجربی را ارائه می کند. فصل ششم به جمع بندی یافته ها، محدودیت ها و پیشنهادهایی برای کارهای آتی خواهد پرداخت.

فصل 2

مفاهیم اولیه

این فصل مفاهیم پایه‌ای مورد نیاز برای درک مسئله و روش پیشنهادی پایان‌نامه را ارائه می‌کند. تمرکز اصلی بر پشته‌ی خودرانی، رایانش لبه‌ی خودرویی، بارسپاری وظایف، شبکه‌ی V2X، یادگیری تقویتی عمیق، الزامات بی‌درنگ و تحمل‌پذیری اشکال است.

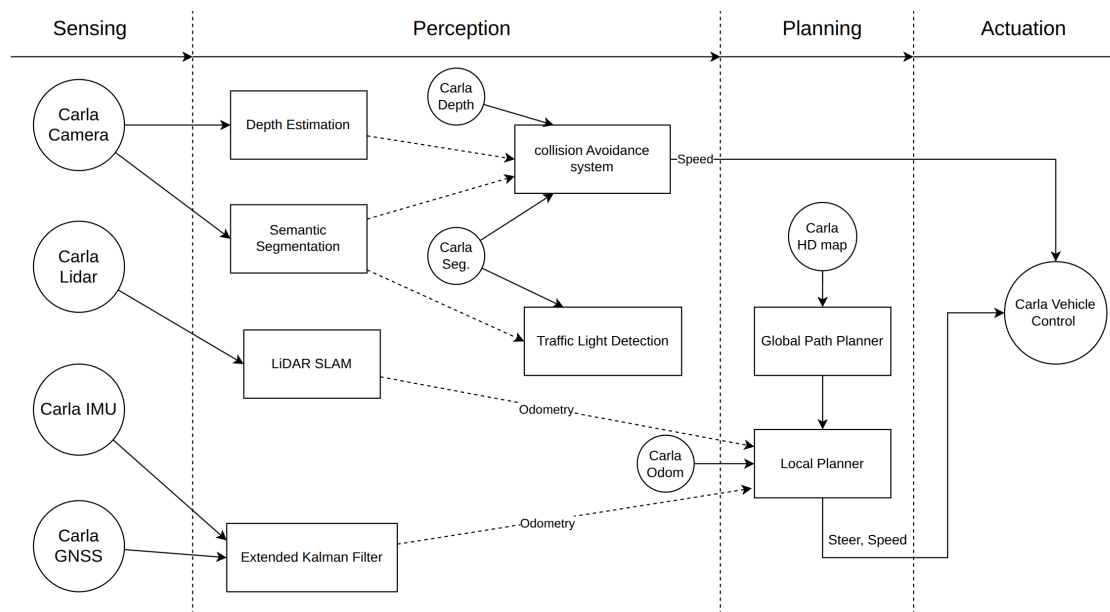
1-2 دوقلوی دیجیتال و ارزیابی حلقه‌بسته

برای تحلیل واقع‌گرایانه‌ی رفتار سامانه، استفاده از یک بستر دوقلوی دیجیتال اهمیت دارد؛ بستری که در آن پویایی خودرو، ترافیک و شبکه به‌صورت هم‌زمان بازنمایی شود. در این پایان‌نامه شبیه‌سازی در محیط CARLA انجام می‌شود [19]. این کار باعث می‌شود تصمیم‌های بارسپاری، نه در یک محیط آزمایشگاهی ایستا، بلکه در شرایط نزدیک به اجرا ارزیابی شوند. به این ترتیب می‌توان بررسی کرد که کل سیستم خودرانی، همراه با بارسپاری و قابلیت اطمینان، به‌صورت یکپارچه چگونه کار می‌کند.

2-2 پشته‌ی نرم‌افزاری خودروهای خودران

یک خودروی خودران را می‌توان به‌عنوان یک سامانه‌ی سایبر-فیزیکی در نظر گرفت که در آن زنجیره‌ی «حسگری، ادراک، تصمیم‌گیری و کنترل» به‌صورت پیوسته اجرا می‌شود. پشته‌ی نرم‌افزاری خودرو این زنجیره را به‌صورت یک خط لوله‌ی پایان‌به‌پایان پیاده‌سازی می‌کند که مجموعه‌ای از وظایف هم‌زمان را روی هم اجرا می‌کند. شکل 1-2 نمای کلی لایه‌های حسگری، ادراک، برنامه‌ریزی¹ و عملگر را در پشته‌ی

¹planning



شکل 2-1: معماری پشته‌ی نرم‌افزاری خودرانی پروژه. فلش‌های نقطه‌چین ارتباطاتی هستند که قطع شده‌اند تا خروجی وظایف نرم روی وظایف سخت اثر نگذارد و این‌گونه نرم محسوب شوند.

این پروژه نشان می‌دهد. هسته‌ی اولیه‌ی این پشته و اتصال CARLA به ROS 2 بر پایه‌ی یک مخزن متن‌باز توسعه یافته است [26]. در ادامه، نقش هر لایه در این خط لوله توضیح داده می‌شود.

حسگری. دوربین و لیدار² داده‌ی خام محیط را تأمین می‌کنند. دوربین تصاویر رنگی و بافت بصری صحنه را فراهم می‌کند و لیدار با اندازه‌گیری فاصله‌ی نقاط، نمای سه‌بعدی اطراف خودرو را باز می‌گرداند. خروجی این حسگرها ورودی مستقیم لایه‌ی ادراک است.

ادراک. وظایف ادراک داده‌ی خام حسگر را به نمایش سطح بالای محیط تبدیل می‌کنند. در پشته‌ی مورد بررسی این پایان‌نامه، این لایه شامل وظایف قطعه‌بندی معنایی، تخمین عمق، تشخیص چراغ راهنمایی، نقشه‌برداری LIO-SAM و مکان‌یابی GNSS/IMU است. این وظایف معمولاً پرمصرف‌ترین بخش خط لوله‌اند و برای بارگذاری مناسبند.

برنامه‌ریزی. این لایه مسیر حرکت خودرو را تعیین می‌کند و شامل وظیفه‌ی «برنامه‌ریزی مسیر» است. کنترل. لایه‌ی کنترل فرمان‌های حرکتی نزدیک به زمان را تولید و اعمال می‌کند. این لایه شامل برنامه‌ریز محلی است که به دو بخش کنترل طولی و کنترل عرضی³ تقسیم می‌شود.

ایمنی. لایه‌ی ایمنی وظیفه‌ی پایش خطرات محیطی و محدودسازی رفتار خودرو را بر عهده دارد تا

²Light Detection and Ranging (LiDAR)

³lateral control

حرکت خودرو در شرایط بحرانی نیز قابل اعتماد بماند. در این پروژه، وظایف ایمنی اصلی «سامانه‌ی اجتناب از برخورد (CAS)⁴» است.

عملگر. لایه‌ی عملگر فرمان‌های نهایی کنترل و ایمنی را روی خودرو اعمال می‌کند. در این پروژه، عملگر شبیه‌ساز فرمان‌های زاویه فرمان، گاز و ترمز را در محیط CARLA اجرا می‌کند و حلقه‌ی کنترل را روی خودرو می‌بندد.

جدول 1-2 وظایف اصلی پیاده‌سازی شده در پشته‌ی نرم‌افزاری این پروژه را بر اساس لایه‌ی پردازشی، مهلت زمانی و نوع قید زمانی خلاصه می‌کند. مهلت‌ها بر اساس محک⁵ Chauffeur تعیین شده‌اند [27] و بر حسب میلی‌ثانیه گزارش می‌شوند. فرکانس هر وظیفه داخل ROS 2 برابر رابطه‌ی زیر تعیین شده است:

$$f = \frac{1}{D} \quad (1-2)$$

که در آن f نرخ اجرای هر وظیفه و D مهلت زمانی آن است.

1-2-2 قطعه‌بندی معنایی

این وظیفه تصویر ورودی را به رده‌های معنایی سطح پیکسل تبدیل می‌کند. برای تولید نقشه‌ی معنایی محیط از مدل SegFormer استفاده می‌شود [28]. خروجی این مدل رده‌ی هر پیکسل را مشخص می‌کند و برای تشخیص ناحیه‌ی امن حرکت و تحلیل صحنه به کار می‌رود.

2-2-2 تخمین عمق

این وظیفه فاصله‌ی نسبی اجسام صحنه را از روی تصویر تخمین می‌زند. برای برآورد عمق تک‌تصویری از مدل Depth Anything استفاده می‌شود [29]. نقشه‌ی عمق برای تخمین فاصله‌ی موانع و تکمیل اطلاعات معنایی مسیر استفاده می‌شود.

3-2-2 تشخیص چراغ راهنمایی

این وظیفه وضعیت رنگ چراغ را از روی تصویر RGB و ماسک قطعه‌بندی معنایی تشخیص می‌دهد [26]. مدل استفاده‌شده در پیاده‌سازی پروژه، یک روش قاعده‌محور مبتنی بر HSV¹² است: ابتدا ناحیه‌ی چراغ با

⁴Collision Avoidance System

⁵benchmark

¹²Hue, Saturation, Value

جدول 2-1: وظایف پیاده‌سازی شده در پشته‌ی خودرانی پروژه

نام وظایف	لایه	شرح مختصر	مهلت (ms)	نوع بی‌درنگ
دوربین RGB، لیدار و IMU	حسگری	تأمین تصویر، نقشه‌ی معنایی، عمق و ابرنقطه از شبیه‌ساز به‌همراه اینرسی	50	سخت
اودومتری ⁶ و وضعیت خودرو	حسگری	سرعت و موقعیت خودرو برای برنامه‌ریزی و ایمنی	50	سخت
قطعه‌بندی معنایی ⁷	ادراک	تعیین رده‌ی ⁸ هر پیکسل	100	نرم
تخمین عمق ⁹	ادراک	برآورد عمق هر پیکسل	100	نرم
تشخیص چراغ راهنمایی	ادراک	شناسایی وضعیت چراغ در مسیر	100	نرم
نقشه‌برداری LIO- SAM	ادراک	تخمین موقعیت و نگاشت محیط با لیدار و IMU	150	سخت
مکان‌یابی GNSS/IMU	ادراک	تلفیق اندازه‌گیری‌های اینرسی و ماهواره‌ای با EKF ¹⁰ برای تخمین موقعیت و اودومتری	50	سخت
برنامه‌ریزی مسیر	برنامه‌ریزی	تولید نقاط مسیر سراسری تا مقصد	رویدادمحور	نرم
برنامه‌ریز محلی	کنترل	کنترل طولی با PID و کنترل عرضی با Stanley	50	سخت
پایشگر ¹¹ برخورد (CAS)	ایمنی	کاهش سرعت یا توقف برای جلوگیری از برخورد	100	سخت
عملگر شبیه‌ساز	عملگر	اعمال فرمان نهایی روی خودرو در CARLA	50	سخت

رنگ معنایی چراغ در تصویر قطعه‌بندی پیدا می‌شود، سپس تصویر چراغ به سه نوار عمودی «قرمز/زرد/سبز» تقسیم می‌شود و برای هر نوار یک امتیاز روشنایی-رنگ محاسبه می‌شود. به‌طور مشخص، برای نوار قرمز امتیاز رنگ از غلبه‌ی مؤلفه‌ی قرمز بر سبز/آبی به‌دست می‌آید:

$$s_{\text{red}} = \text{mean}(\max(R - \max(G, B), 0)) \quad (2-2)$$

و امتیاز نهایی هر رنگ از ضرب روشنایی در امتیاز رنگ محاسبه می‌شود. رنگ با بیشترین امتیاز معتبر به‌عنوان وضعیت چراغ انتخاب می‌شود؛ در غیر این صورت خروجی UNKNOWN گزارش می‌شود.

4-2-2 نقشه برداری LiDAR SLAM¹³

در این پایان نامه برای نگاشت محیط و تخمین وضعیت از LIO-SAM مبتنی بر لیدار/IMU استفاده می شود [30]. صورت بندی¹⁴ این روش بر پایه ی تخمین حالت در قالب گراف فاکتورها است و از ترکیب قیدهای اینرسی، لیدار، GPS (در صورت دسترس پذیری) و بستن حلقه استفاده می کند.

مدل حالت و دستگاه مختصات. در LIO-SAM، چارچوب جهانی با W و چارچوب بدنه/حسگر با B نشان داده می شود و حالت ربات در زمان k به صورت

$$\mathbf{x}_k = \begin{bmatrix} \mathbf{R}_k^T & \mathbf{p}_k^T & \mathbf{v}_k^T & \mathbf{b}_k^T \end{bmatrix}^T \quad (3-2)$$

تعریف می شود که در آن $\mathbf{R}_k \in SO(3)$ ماتریس دوران، $\mathbf{p}_k$ موقعیت، $\mathbf{v}_k$ سرعت، و $\mathbf{b}_k$ بایاس IMU است. **صورت بندی MAP¹⁵ و گراف فاکتورها.** هدف، یافتن دنباله ی حالتی است که با اندازه گیری های چند حسگر بیشترین سازگاری را داشته باشد. با فرض نویز گاوسی، مسئله ی MAP به کمینه سازی مربعات وزن دار تبدیل می شود:

$$\mathbf{X}^* = \arg \min_{\mathbf{X}} \sum_i \|\mathbf{r}_i(\mathbf{X})\|_{\Sigma_i^{-1}}^2 \quad (4-2)$$

که در آن $\mathbf{X}$ مجموعه ی گره های حالت، $\mathbf{r}_i$ باقیمانده ی فاکتور i ام، و Σ_i کوواریانس همان فاکتور است. در LIO-SAM چهار نوع فاکتور اصلی تعریف می شود: (۱) فاکتور پیش انتگرال گیری IMU، (۲) فاکتور اودومتري لیدار، (۳) فاکتور GPS، (۴) فاکتور بستن حلقه [30].

فاکتور IMU و پیش انتگرال گیری. اندازه گیری های خام ژيروسکوپ و شتاب سنج به صورت زیر مدل می شوند:

$$\hat{\boldsymbol{\omega}}_t = \boldsymbol{\omega}_t + \mathbf{b}_t^\omega + \mathbf{n}_t^\omega, \quad \hat{\mathbf{a}}_t = \mathbf{R}_t^{BW}(\mathbf{a}_t - \mathbf{g}) + \mathbf{b}_t^a + \mathbf{n}_t^a \quad (5-2)$$

و دینامیک حالت گسسته با گام Δt به صورت

$$\mathbf{v}_{t+\Delta t} = \mathbf{v}_t + \mathbf{g}\Delta t + \mathbf{R}_t(\hat{\mathbf{a}}_t - \mathbf{b}_t^a - \mathbf{n}_t^a)\Delta t \quad (6-2)$$

$$\mathbf{p}_{t+\Delta t} = \mathbf{p}_t + \mathbf{v}_t\Delta t + \frac{1}{2}\mathbf{g}\Delta t^2 + \frac{1}{2}\mathbf{R}_t(\hat{\mathbf{a}}_t - \mathbf{b}_t^a - \mathbf{n}_t^a)\Delta t^2 \quad (7-2)$$

$$\mathbf{R}_{t+\Delta t} = \mathbf{R}_t \exp((\hat{\boldsymbol{\omega}}_t - \mathbf{b}_t^\omega - \mathbf{n}_t^\omega)\Delta t) \quad (8-2)$$

¹³Simultaneous Localization and Mapping (SLAM)

¹⁴formulation

¹⁵Maximum A Posteriori

بیان می‌شود. در روابط بالا، $\hat{\mathbf{a}}_t$ و $\hat{\boldsymbol{\omega}}_t$ به ترتیب اندازه‌گیری ژيروسکوپ و شتاب‌سنج، $\boldsymbol{\omega}_t$ سرعت زاویه‌ای واقعی، $\mathbf{a}_t$ شتاب واقعی، $\mathbf{g}$ بردار گرانش، $\mathbf{b}_t^\omega$ ، $\mathbf{b}_t^a$ بایاس ژيروسکوپ/شتاب‌سنج، $\mathbf{n}_t^\omega$ ، $\mathbf{n}_t^a$ نویز اندازه‌گیری، $\mathbf{R}_t^{BW}$ دوران از چارچوب جهانی به بدنه، و Δt گام زمانی گسسته است. برای کارایی بی‌درنگ، LIO-SAM از پیش‌انتگرال‌گیری IMU بین دو گره i, j استفاده می‌کند تا قید نسبی $\Delta \mathbf{R}_{ij}$ ، $\Delta \mathbf{v}_{ij}$ ، $\Delta \mathbf{p}_{ij}$ به دست آید:

$$\Delta \mathbf{v}_{ij} = \mathbf{R}_i^T (\mathbf{v}_j - \mathbf{v}_i - \mathbf{g} \Delta t_{ij}), \quad \Delta \mathbf{p}_{ij} = \mathbf{R}_i^T \left(\mathbf{p}_j - \mathbf{p}_i - \mathbf{v}_i \Delta t_{ij} - \frac{1}{2} \mathbf{g} \Delta t_{ij}^2 \right) \quad (9-2)$$

$$\Delta \mathbf{R}_{ij} = \mathbf{R}_i^T \mathbf{R}_j. \quad (10-2)$$

این قیود به صورت فاکتور بین دو گرهی متوالی وارد بهینه‌سازی می‌شوند.

فاکتور اودومتری لیدار. پس از استخراج ویژگی‌های لبه‌ای/صفحه‌ای، تنها قاب‌کلیدی‌ها¹⁶ (فریم‌های کلیدی منتخب) وارد گراف می‌شوند تا تراکم گراف و هزینه‌ی محاسباتی کنترل شود. برای قاب‌کلیدی جدید، تطبیق اسکن در یک پنجره‌ی محلی از زیرقاب‌کلیدی‌ها انجام می‌گیرد و باقیمانده‌های هندسی لبه و صفحه ساخته می‌شوند. هدف این مرحله، یافتن پوز فریم جدید است به طوری که مجموع خطاهای هندسی نقطه-به-لبه و لبه-به-نقطه-به-صفحه کمینه شود. فرم کلی کمینه‌سازی لیدار به صورت

$$\min_{\mathbf{T}_{i+1}} \left(\sum_{k \in \mathcal{E}} d_k^e(\mathbf{T}_{i+1}) + \sum_{k \in \mathcal{P}} d_k^p(\mathbf{T}_{i+1}) \right) \quad (11-2)$$

است که معمولاً با گاوس-نیوتن حل می‌شود [30]. خروجی این مرحله، تبدیل نسبی بین دو گرهی متوالی است:

$$\Delta \mathbf{T}_{i,i+1} = \mathbf{T}_i^T \mathbf{T}_{i+1} \quad (12-2)$$

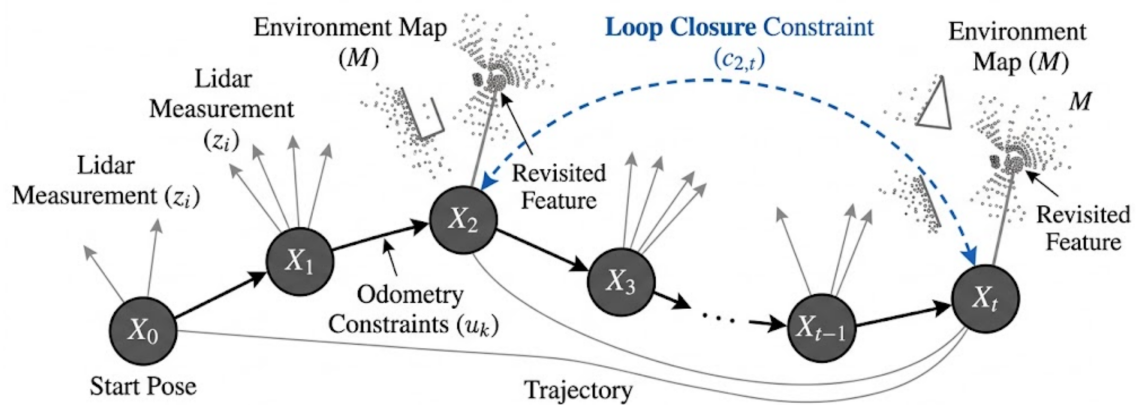
و همین تبدیل به عنوان فاکتور اودومتری لیدار وارد گراف می‌شود.

بستن حلقه¹⁷. در این مرحله، سامانه با استفاده از شباهت هندسی ویژگی‌های لیدار تشخیص می‌دهد که به یک ناحیه‌ی قبلاً دیده‌شده بازگشته است؛ سپس یک قید نسبی بین گرهی جاری و گرهی متناظر گذشته در گراف اضافه می‌شود. این قید باعث می‌شود ناسازگاری مسیر در بازگشت به همان مکان کاهش یابد و رانش تجمعی¹⁸ اصلاح شود. در نهایت، با بهینه‌سازی افزایشی گراف (مانند iSAM2)، تخمین مسیر و نقشه به صورت بی‌درنگ به روزرسانی می‌شود [30].

¹⁶Keyframe

¹⁷Loop Closure

¹⁸Long-term drift



شکل 2-2: شماتیک قید بستن حلقه در گراف فاکتورها: پس از بازدید مجدد یک ویژگی محیطی، قید نسبی بین گره‌ی جاری و یک گره‌ی قبلی افزوده می‌شود تا رانش تجمعی مسیر کاهش یابد.

5-2-2 مکان‌یابی GNSS/IMU

مسیر جایگزین مکان‌یابی با تلفیق GNSS/IMU و فیلتر کالمن توسعه‌یافته (EKF) پیاده‌سازی شده است [26]. فرم استاندارد پیش‌بینی و به‌روزرسانی به‌صورت زیر است:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_k), \quad \mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k)^{-1} \quad (13-2)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})) \quad (14-2)$$

که در آن $\hat{\mathbf{x}}$ بردار حالت و $\mathbf{K}_k$ بهره‌ی کالمن¹⁹ است.

6-2-2 برنامه‌ریزی مسیر

در این وظیفه، مسیر کلی خودرو با استفاده از HD map²⁰ از مبدأ تا مقصد تعیین می‌شود و دنباله‌ای از نقاط مسیر برای کنترل محلی منتشر می‌شود [26]. این وظیفه ماهیت رویدادمحور دارد و با تغییر هدف یا وضعیت صحنه بازتولید می‌شود.

7-2-2 برنامه‌ریز محلی

برنامه‌ریز محلی مسیر سراسری را دنبال می‌کند و فرمان نزدیک‌به‌زمان را در دو کانال مستقل تولید می‌کند: کنترل طولی برای گاز/ترمز و کنترل عرضی برای زاویه فرمان. در پیاده‌سازی جاری، کنترل طولی با PID و

¹⁹Kalman gain

²⁰High-Definition Map

کنترل عرضی با کنترلر Stanley انجام می‌شود [3, 26]..

کنترل طولی. هدف این کانال رساندن سرعت فعلی خودرو v به سرعت مرجع v_{ref} است؛ v_{ref} برابر کمینه‌ی سرعت کروز و سقف ایمنی صادرشده از CAS در نظر گرفته می‌شود. خطای سرعت برابر است با

$$e_v(t) = v_{\text{ref}}(t) - v(t) \quad (15-2)$$

و فرمان گاز با کنترلر تناسبی-انتگرالی-مشتقی

$$u_{\text{lon}}(t) = K_p e_v(t) + K_i \int_0^t e_v(\tau) d\tau + K_d \frac{de_v(t)}{dt} \quad (16-2)$$

تولید می‌شود. خروجی در بازه‌ی $[0, 1]$ اشباع می‌شود تا با ورودی گاز شبیه‌ساز سازگار باشد.

مدل دوچرخه. کنترل عرضی بر مدل سینماتیکی دوچرخه²¹ استوار است. اگر (x, y) موقعیت، ψ سمت خودرو، v سرعت طولی، δ زاویه فرمان چرخ جلو و L فاصله‌ی محورها باشد، دینامیک صفحه‌ای خودرو به صورت

$$\dot{x} = v \cos \psi, \quad \dot{y} = v \sin \psi, \quad \dot{\psi} = \frac{v}{L} \tan \delta \quad (17-2)$$

نوشته می‌شود. این مدل پیوند میان زاویه فرمان و نرخ چرخش سمت را نشان می‌دهد و مبنای قانون Stanley است [3].

کنترل عرضی با Stanley. کنترلر Stanley²² خطای سمت²³ و خطای عرضی مسیر²⁴ را هم‌زمان جبران می‌کند. هندسه‌ی این قانون در شکل 2-3 آمده است: نقطه‌ی مرجع محور جلو است، e فاصله‌ی عمودی محور جلو تا مسیر، ψ در شکل همان خطای سمت ψ_e ، δ زاویه‌ی فرمان چرخ جلو، L فاصله‌ی محورها و v_f سرعت محور جلو است. اگر ψ_p سمت مماس مسیر در نقطه‌ی تصویر خودرو روی چندخطی نقاط مسیر باشد، خطای سمت برابر است با

$$\psi_e = \psi_p - \psi. \quad (18-2)$$

اگر e فاصله‌ی علامت‌دار محور جلو تا مسیر باشد (مثبت وقتی خودرو در سمت چپ مسیر قرار دارد)، قانون فرمان

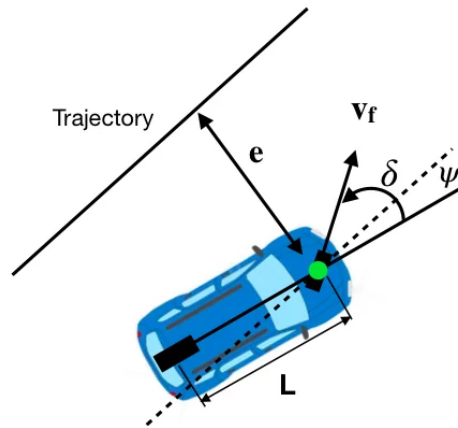
$$\delta = \psi_e + \arctan\left(\frac{k e}{v + k_s}\right) \quad (19-2)$$

²¹bicycle model

²²Stanley controller

²³heading error

²⁴cross-track error



شکل 2-3: هندسه‌ی کنترلر Stanley: خطای عرضی e از محور جلو تا مسیر، خطای سمت (در شکل ψ و در متن ψ_e)، زاویه‌ی فرمان δ ، فاصله‌ی محورها L و سرعت محور جلو v_f [3].

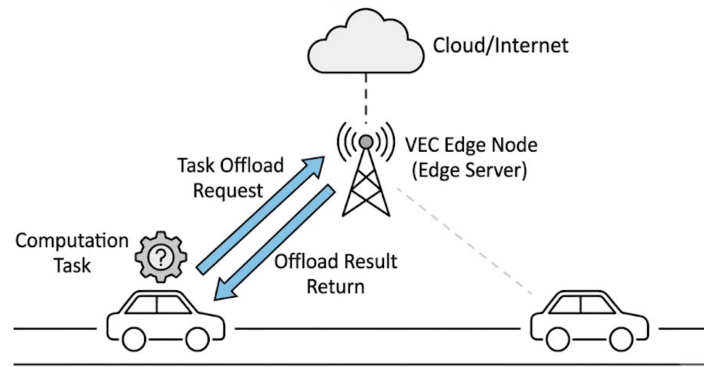
است؛ که $k > 0$ بهره‌ی خطای عرضی و $k_s > 0$ جمله‌ی نرم‌سازی برای جلوگیری از تقسیم بر صفر در سرعت نزدیک به صفر است. جمله‌ی نخست خودرو را با مسیر هم‌راستا می‌کند و جمله‌ی دوم آن را به روی مسیر برمی‌گرداند؛ با افزایش v ، سهم خطای عرضی کوچک‌تر می‌شود تا فرمان در سرعت بالا ناپایدار نشود. در پیاده‌سازی، δ به بازه‌ی فرمان شبیه‌ساز محدود و نرخ تغییر آن نیز سقف‌گذاری می‌شود.

8-2-2 پایشگر برخورد

در پیاده‌سازی کد، خروجی CAS در هر تیک زمانی ($T = 0/1s$) یک فرمان سرعت بر حسب km/h تولید می‌کند که بر اساس وضعیت خطر به صورت چندبخشی تعریف می‌شود. اگر v_k سرعت فعلی خودرو، $\mathbb{K}_{col}$ پرچم «خطر برخورد قطعی» و $\mathbb{K}_{zone}$ پرچم «وجود شیء در ناحیه‌ی برخورد» باشد، فرمان نسخه‌ی CAS برابر است با:

$$v_{k+1}^{cas} = \begin{cases} 0, & \mathbb{K}_{col} = 1 \\ \max(0, v_k - 1), & \mathbb{K}_{col} = 0 \wedge \mathbb{K}_{zone} = 1 \\ \min(v_{max}, v_k + 1), & \mathbb{K}_{col} = 0 \wedge \mathbb{K}_{zone} = 0 \end{cases} \quad (20-2)$$

که در آن حد بالای سرعت برابر 50 km/h است. بنابراین، در خطر قطعی فرمان توقف کامل صادر می‌شود؛ در حضور شیء داخل ناحیه خطر، سرعت به صورت پله‌ای کاهش می‌یابد؛ و در نبود خطر، سرعت تا سقف مجاز به صورت پله‌ای افزایش می‌یابد.



شکل 2-4: مفهوم بارسپاری محاسباتی در رایانش لبه خودرویی.

منطق تشخیص خطر نیز خروجی قطعه‌بندی معنایی و تخمین عمق را به‌عنوان ورودی می‌گیرد. ابتدا فاصله‌ی توقف از سرعت فعلی تخمین زده می‌شود:

$$d_{\text{stop}}(v) = \max(d_{\min}, \alpha v^2 + \beta v) \quad (21-2)$$

که در آن $d_{\min}$ حداقل فاصله‌ی امن، α ضریب ترمزی (مربعی با سرعت)، و β ضریب فاصله‌ی ناشی از تأخیر حسگر/عملگر است. در این پیاده‌سازی مقادیر $d_{\min} = 3 \text{ m}$ ، $\alpha = 0/004286$ و $\beta = 0/3$ در نظر گرفته شده‌اند. سپس ماسک عمق با شرط $d < d_{\text{stop}}$ و ماسک رده‌های مانع (خودرو/عابر/...) ترکیب می‌شود؛ اگر پیکسل معتبر داخل چندضلعی ناحیه‌ی خطر وجود داشته باشد، پرچم خطر فعال می‌شود.

3-2 رایانش لبه و بارسپاری وظایف

رایانش لبه خودرویی برای کاهش بار پردازشی محلی، بخشی از وظایف را به زیرساخت نزدیک (لبه/ابر نزدیک) منتقل می‌کند. در این پایان‌نامه، بارسپاری به‌عنوان یک تصمیم پویا در زمان اجرا دیده می‌شود؛ یعنی سامانه باید در هر لحظه تشخیص دهد اجرای محلی مناسب‌تر است یا اجرای بارسپاری‌شده. شکل 2-4 نمای مفهومی این فرآیند را نشان می‌دهد: خودرو وظیفه‌ی محاسباتی را به گره‌ی لبه‌ی VEC می‌فرستد و نتیجه‌ی پردازش را دریافت می‌کند؛ گره‌ی لبه نیز می‌تواند به ابر/Internet متصل باشد.

مزیت اصلی بارسپاری، کاهش مصرف انرژی سکوی داخلی و بهبود استفاده از منابع محاسباتی است؛ اما این مزیت تنها زمانی حفظ می‌شود که تأخیر شبکه و شرایط ترافیکی کنترل شود. اگر تأخیر رفت و برگشت افزایش یابد، کیفیت سرویس وظیفه‌ی بارسپاری‌شده ممکن است کم شود [5, 6, 8].

زمان اجرای انتها به انتها²⁵ برای هر وظیفه به صورت زیر تعریف می شود:

$$L = \begin{cases} t_{\text{local}} + t_{\text{decider}}, & \text{اگر وظیفه بار سپاری نشود} \\ t_{\text{server}} + t_{\text{decider}} + t_{\text{tx}}, & \text{اگر وظیفه بار سپاری شود} \end{cases} \quad (22-2)$$

که در آن t_{local} زمان پردازش محلی، t_{server} زمان پردازش روی سرور، t_{decider} زمان تصمیم گیرنده، و t_{tx} تأخیر ارسال است. با فرض برابری فرکانس پردازنده ی سرور و خودرو داریم

$$f_{\text{server}} = f_{\text{local}} \quad (23-2)$$

و در نتیجه

$$t_{\text{server}} = t_{\text{local}}. \quad (24-2)$$

در این پایان نامه، کیفیت سرویس به صورت زیر تعریف می شود [2]:

$$QoS_i(\tau_j) = \begin{cases} 1, & L_i(\tau_j) < T_j \\ \frac{q \times T_j - L_i(\tau_j)}{(q-1) \times T_j}, & T_j \leq L_i(\tau_j) \leq q \times T_j \\ 0, & q \times T_j < L_i(\tau_j) \end{cases} \quad (25-2)$$

که در آن $L_i(\tau_j)$ تأخیر اجرای وظیفه ی τ_j روی پردازنده ی i ، T_j مهلت زمانی وظیفه، و q ضریب نرم سازی²⁶ مهلت است. در این پایان نامه مقدار $q = 2$ در نظر گرفته شده است.

انرژی مصرفی پردازش محلی روی خودرو بر اساس مدل تلفات توان مدارهای CMOS محاسبه می شود [4]:

$$E_{\text{local}} = \kappa \cdot f^2 \cdot c_j \quad (26-2)$$

که در آن κ ثابت معماری تراشه، f فرکانس پردازنده ی محلی خودرو، و c_j تعداد سیکل های محاسباتی وظیفه ی τ_j است. در این پروژه فرکانس پردازنده ی خودرو به صورت یکنواخت از بازه ی [1, 2] GHz نمونه برداری می شود [31].

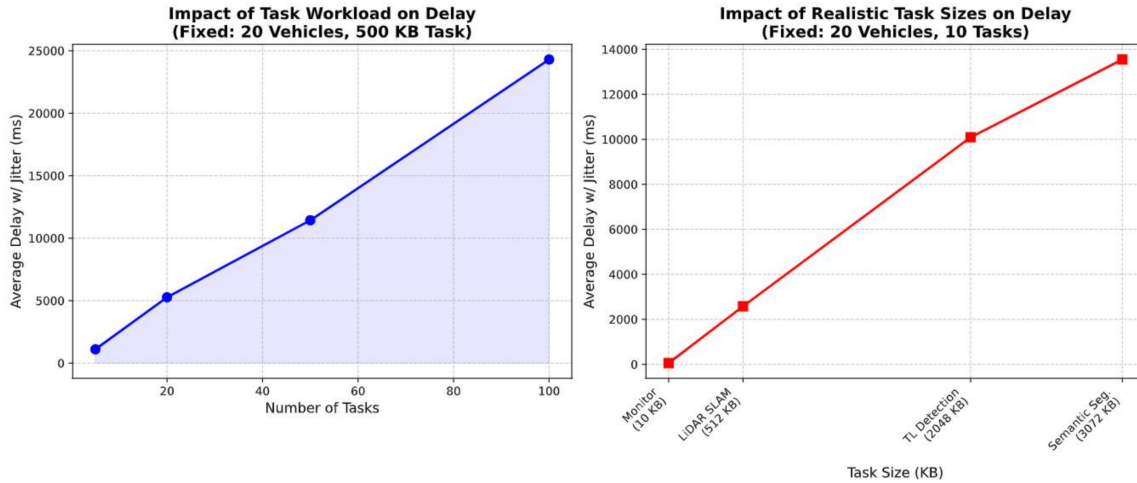
علاوه بر آن، انرژی ارسال²⁷ به صورت زیر تعریف می شود [4]:

$$E_{\text{tx}} = p \cdot L_i^t(\tau_j) + k \cdot \mu_j \cdot \frac{c_j}{f} \quad (27-2)$$

²⁵execution time

²⁶softening factor

²⁷transmission energy



شکل 2-5: تأثیر بار و اندازه‌ی وظایف بر تأخیر میانگین شبکه: (چپ) افزایش تأخیر با تعداد وظایف و (راست) افزایش تأخیر با اندازه‌ی واقعی وظایف ادراکی.

که در آن p توان ارسال، $L_i^t(\tau_j)$ تأخیر انتقال داده به سرور لبه، k توان پردازش محلی، μ_j سهم پیش پردازش محلی وظیفه، c_j تعداد سیکل‌های محاسباتی وظیفه، و f فرکانس پردازنده‌ی محلی است. در نتیجه، انرژی کل وظیفه‌ی τ_j به صورت زیر محاسبه می‌شود:

$$E = \begin{cases} E_{\text{local}} + E_{\text{decider}}, & \text{اگر وظیفه بارسپاری نشود} \\ E_{\text{tx}} + E_{\text{decider}}, & \text{اگر وظیفه بارسپاری شود} \end{cases} \quad (28-2)$$

که در آن E_{decider} انرژی تصمیم‌گیرنده برای انتخاب محل اجرا است.

4-2 شبکه‌ی V2X و اثر آن بر تصمیم بارسپاری

V2X بستر تبادل داده بین خودرو، زیرساخت و سایر گره‌های شبکه است. در سناریوهای واقعی، تأخیر V2X ثابت نیست و با چگالی ترافیک، ازدحام لینک، و شرایط لحظه‌ای شبکه تغییر می‌کند. به همین دلیل، سیاست بارسپاری مؤثر باید آگاه از وضعیت شبکه باشد، نه صرفاً آگاه از بار محاسباتی. در این پایان‌نامه، تأخیر شبکه به عنوان ورودی تصمیم‌گیری و ارزیابی عملیاتی سامانه در حلقه‌ی بسته لحاظ می‌شود. شکل 2-5 اثر تعداد و اندازه‌ی وظایف را بر تأخیر میانگین شبکه نشان می‌دهد.

5-2 یادگیری تقویتی عمیق برای تصمیم بارسپاری

در محیط‌های پویای سایبر-فیزیکی، تصمیم بارسپاری باید با تغییرات لحظه‌ای شبکه و بار محاسباتی سازگار باشد. یادگیری تقویتی عمیق²⁸ با ترکیب شبکه‌های عصبی و تصمیم‌گیری مارکوف، رویکردی مؤثر برای این مسئله است [7]. الگوریتم‌های جدولی مانند Q-یادگیری در فضای حالت-عمل بزرگ با نفرین ابعاد²⁹ مواجه می‌شوند؛ به همین دلیل به جای جدول Q، از تقریب‌گر تابع ارزش مبتنی بر شبکه‌ی عصبی با پارامتر θ استفاده می‌شود:

$$Q(s, a) \approx Q(s, a; \theta). \quad (29-2)$$

مسئله در قالب فرآیند تصمیم مارکوف³⁰ مدل می‌شود. هر وظیفه با حجم داده، تعداد سیکل محاسباتی و مهلت زمانی مشخص می‌شود. در گام t ، ایجنت مشاهده‌ی O_t را دریافت می‌کند، عمل a_t را برمی‌گزیند، پاداش R_t را می‌گیرد و به O_{t+1} می‌رود. هدف، بیشینه‌سازی پاداش تجمعی تنزیل یافته است:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid s_0 = s \right], \quad (30-2)$$

که در آن $\gamma \in (0, 1)$ عامل تنزیل است.

فضای مشاهده پس از نرمال‌سازی Min-Max به صورت زیر ساخته می‌شود:

$$O_t = [D_1, \dots, D_{Z^2}, \tau_1, \dots, \tau_N, f_{\text{norm}}, x_{\text{norm}}, y_{\text{norm}}], \quad (31-2)$$

که در آن D_k چگالی ناحیه‌ی k ، و τ_i شامل اندازه‌ی داده، سیکل و مهلت وظیفه‌ی i است. با $Z = 2$ و $N = 3$ ، بعد مشاهده برابر ۱۶ است. فضای عمل برای هر وظیفه باینری است (0 محلی و 1 بارسپاری) و اندازه‌ی آن برابر $|A| = 2^N$ می‌شود؛ بنابراین برای $N = 3$ داریم $|A| = 8$.

تابع پاداش³¹ مصالحه‌ی انرژی و تأخیر را با وزن α و جریمه‌ی نقض مهلت مدل می‌کند:

$$R_t = - \left(\alpha \frac{E_{\text{total}}}{E_{\text{max}}} + (1 - \alpha) \frac{D_{\text{total}}}{D_{\text{max}}} + \frac{1/3^m}{1/3^3} \right), \quad (32-2)$$

که در آن E_{total} انرژی محلی، D_{total} تأخیر کل (شامل تأخیر شبکه) و m تعداد وظایف نقض کننده‌ی مهلت است. سیاست مستقر با $\alpha = 0/6$ آموزش دیده است.

²⁸Deep Reinforcement Learning (DRL)

²⁹curse of dimensionality

³⁰Markov Decision Process (MDP)

³¹reward function

برای غلبه بر بیش‌برآورد ارزش در DQN کلاسیک، از شبکه‌ی Q دوگانه³² استفاده می‌شود [7]. شبکه‌ی اصلی با پارامتر θ اقدام را انتخاب می‌کند و شبکه‌ی هدف با پارامتر θ^- آن را ارزیابی می‌کند:

$$Y_t^{\text{DDQN}} = R_{t+1} + \gamma Q(O_{t+1}, \arg \max_{a'} Q(O_{t+1}, a'; \theta); \theta^-). \quad (33-2)$$

تجربیات $(O_t, a_t, R_{t+1}, O_{t+1})$ در حافظه‌ی بازپخش ذخیره و به‌صورت تصادفی نمونه‌برداری می‌شوند تا همبستگی زمانی کاهش یابد. تابع زیان میانگین مربعات خطا روی نمونه‌ها کمینه می‌شود و وزن‌های θ با بهینه‌ساز Adam به‌روز می‌گردند. در این پایان‌نامه، شبکه‌ی Q یک پرسپترون چندلایه با ورودی ۱۶ بعدی و خروجی ۸ بعدی است که پس از آموزش، به‌صورت قطعی در حلقه‌ی بسته تصمیم‌گیری را صادر می‌کند.

6-2 قابلیت اطمینان و تحمل‌پذیری اشکال

در سامانه‌های خودران، خطای یک وضعیت اجتناب‌ناپذیر است و طراحی باید بر «مدیریت خطا» متمرکز باشد، نه صرفاً «پیشگیری از خطا». اصل کلیدی این پایان‌نامه آن است که بروز خطا نباید به فاجعه منجر شود. سامانه باید بتواند خطا را تشخیص دهد، اثر آن را محدود کند و عملکرد ایمن را تا حد ممکن حفظ نماید.

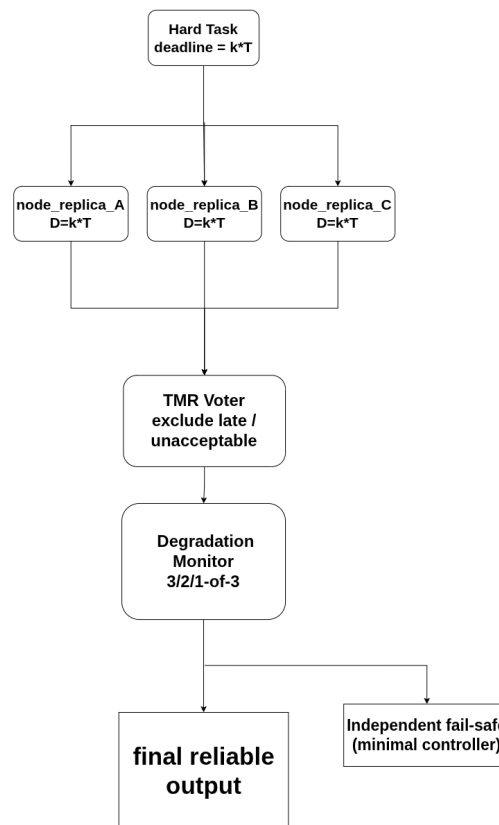
برای وظایف سخت بی‌درنگ، از سازوکار افزونگی TMR (سه‌گانه‌سازی پیمانه) استفاده می‌شود. همان‌طور که در شکل 6-2 دیده می‌شود، وظیفه‌ی سخت با مهلت $D = kT$ به‌صورت موازی روی سه نسخه‌ی مستقل (node_replica_A/B/C) اجرا می‌شود. خروجی هر نسخه به رأی‌گیر TMR می‌رسد. رأی‌گیر دو نوع پالایش انجام می‌دهد: (۱) حذف نسخه‌های دیررس یا خاموش که مهلت را نقض کرده‌اند، و (۲) حذف مقادیر غیرقابل قبول که از نظر عددی با بقیه سازگار نیستند. مدل خطای منطقی در این پایان‌نامه وارونگی بیت³³ است؛ یعنی خروجی یک نسخه خراب می‌شود، در حالی که ممکن است همچنان به‌موقع برسد. در این حالت، رأی‌گیر با مقایسه‌ی روادار مقادیر، نسخه‌ی معیوب را کنار می‌گذارد و از اکثریت سالم خروجی مطمئن می‌سازد.

نرخ وقوع خطا با فرایند پواسون مدل می‌شود؛ احتمال رخداد k خطا در بازه‌ی زمانی t برابر است با

$$P(N(t) = k) = \frac{(\lambda t)^k}{k!} e^{-\lambda t}, \quad (34-2)$$

³²Double Deep Q-Network (DDQN)

³³Bit Flip



شکل 2-6: سازوکار تحمل پذیری اشکال مبتنی بر TMR: اجرای سه گانه ی وظیفه ی سخت، رأی گیر برای حذف خروجی های دیررس/ غیر قابل قبول، پایشگر تنزل و مسیر ایمن حداقلی.

که در آن λ نرخ میانگین خطا در واحد زمان است. در این پایان نامه $\lambda = 0/5$ قرار داده شده و بر اساس همین مدل، نرخ تزریق خطا در ارزیابی قابلیت اطمینان تنظیم می شود.

پس از رأی گیری، پایشگر تنزل³⁴ سطح افزونگی باقی مانده را در حالت های 3-از-3، 2-از-3 یا 1-از-3 ثبت می کند. اگر رأی معتبر به دست آید، خروجی نهایی مطمئن به کنترل کننده ارسال می شود؛ در غیر این صورت سامانه به حالت ایمن حداقلی³⁵ تغییر مسیر³⁶ می دهد تا از رفتار خطرناک جلوگیری شود. این افزونگی هزینه ی محاسباتی و زمانی اضافه دارد؛ بنابراین ارزیابی باید نشان دهد آیا سربار در برابر کاهش ریسک عملکردی توجیه پذیر است یا خیر.

³⁴Degradation Monitor

³⁵Fail-safe

³⁶switch

7-2 الزامات بی‌درنگ سخت و نرم

همه‌ی وظایف خودرو خودران یکسان نیستند. وظایف ایمنی محور در مسیر کنترل، معمولاً بی‌درنگ سخت هستند؛ یعنی دیررسیدن پاسخ می‌تواند به پیامد خطرناک منجر شود. در مقابل، برخی وظایف ادراکی یا کمکی را می‌توان در قالب بی‌درنگ نرم³⁷ مدل کرد؛ یعنی خروجی دیررس ارزش کمتر دارد، اما کاملاً بی‌ارزش نیست.

بر این اساس، در پشته‌ی این پروژه سه وظیفه‌ی ادراکی قطعه‌بندی معنایی، تخمین عمق و تشخیص چراغ راهنمایی به‌عنوان بی‌درنگ نرم دسته‌بندی می‌شوند. سارانیا و همکاران تشخیص مانع و چراغ راهنمایی را با زمان‌بندی بی‌درنگ نرم و مهلت نرم روی لبه‌ی خودرویی ارزیابی کرده‌اند [17]؛ و جیکوبلیس و همکاران نیز استنتاج قطعه‌بندی معنایی صحنه‌ی رانندگی را در قالب کاربردهای بی‌درنگ نرم (با تحمل نقض گاه‌به‌گاه مهلت) بررسی کرده‌اند [16]. همین نگاه در این پایان‌نامه برای تخمین عمق نیز به کار می‌رود؛ زیرا مانند دو وظیفه‌ی دیگر، خروجی دیررس همچنان قابل استفاده است و برخلاف مسیر کنترل سخت، نقض گاه‌به‌گاه مهلت الزاماً به پیامد ایمنی فوری منجر نمی‌شود.

تفکیک بی‌درنگ سخت/نرم برای این پایان‌نامه ضروری است؛ وظایف سخت با سازوکارهای قابلیت اطمینان ایمن نگه داشته می‌شوند و وظایف نرم برای کاهش بار محاسباتی محلی بارسپاری می‌گردند. به بیان دیگر، هر سود انرژی باید با قید ایمنی زمانی ارزیابی شود.

³⁷soft real-time

فصل 3

کارهای پیشین

این فصل کارهای مرتبط با این پایان‌نامه را مرور می‌کند. هدف، مشخص کردن شکاف‌هایی است که پژوهش حاضر با ترکیب تصمیم‌بارسپاری آگاه از شبکه و تحمل‌پذیری اشکال برای وظایف سخت بی‌درنگ پر می‌کند.

1-3 انتخاب سرور و بارسپاری مبتنی بر یادگیری تقویتی

یکی از محورهای اصلی رایانش لبه‌ی سیار¹، انتخاب سرور مناسب و تصمیم‌بارسپاری تحت پویایی کانال و بار محاسباتی است. لیو و کائو انتخاب سرور مبتنی بر یادگیری تقویتی عمیق را بررسی کرده و نشان داده‌اند که سیاست یادگرفته‌شده می‌تواند تأخیر و هزینه‌ی سرویس را نسبت به قواعد حریصانه کاهش دهد [32]. شوئه و همکاران با تخمین پارامترهایی مانند بار و کیفیت لینک، سازوکاری برای پایداری بیشتر کردن تصمیم بارسپاری پیشنهاد کرده‌اند [33]. هو و نگوین نیز بهینه‌سازی مشترک انتخاب سرور، بارسپاری مشارکتی و تحویل را با DRL در شبکه‌های دسترسی چندگانه مطرح کرده‌اند و نشان داده‌اند که تصمیم‌های بارسپاری و تحرک کاربر به هم وابسته‌اند و جداسازی آن‌ها مطلوب نیست [34].

در لایه‌ی تخصیص منابع، الفقیه و همکاران بارسپاری وظیفه و تخصیص منابع را با یادگیری تقویتی مبتنی بر SARSA حل کرده و بر مصالحه‌ی انرژی/تأخیر در گره‌های لبه تأکید کرده‌اند [35]. این دسته از پژوهش‌ها اثربخشی DRL/RL را برای تصمیم‌بارسپاری تأیید می‌کنند، اما معمولاً روی مدل‌های انتزاعی صف/شبکه ارزیابی می‌شوند و اثر تأخیر شبکه در حلقه را روی مسیر ادراک و کنترل خودروی خودران اندازه نمی‌گیرند.

¹Mobile Edge Computing (MEC)

2-3 قابلیت اطمینان و زمان‌بندی استنتاج عمیق در لبه

برای کاربردهای بی‌درنگ، صرف کاهش میانگین تأخیر کافی نیست؛ باید قابلیت اطمینان اتمام وظیفه در مهلت نیز تضمین شود. ما و همکاران زمان‌بندی برخط استنتاج شبکه‌های عصبی عمیق (DNN) را با آگاهی از قابلیت اطمینان در MEC مطرح کرده‌اند که بین مهلت، بار سرور و احتمال موفقیت مصالحه می‌کند [36]. لئو و همکاران نیز شتاب‌دهی استنتاج DNN با تضمین قابلیت اطمینان را در لبه‌ی خودرویی بررسی کرده‌اند تا تحت قیود زمانی و منابع محدود خودرو، زمان اجرا حفظ شود [37].

این آثار قابلیت اطمینان را عمدتاً از زاویه‌ی زمان‌بندی و تخصیص منابع برای استنتاج عمیق می‌بینند. در مقابل، پژوهش حاضر علاوه بر معیارهای زمان اجرای نرم، تحمل‌پذیری اشکال نرم‌افزاری در مسیر کنترل سخت‌بی‌درنگ مانند اجتناب از برخورد با TMR را نیز در همان حلقه‌ی ارزیابی بارسپاری می‌گنجاند؛ مرور آثار مرتبط با تکرار وظیفه و افزونگی سخت‌افزاری در ادبیات موضوع فصل اول آمده است.

3-3 بارسپاری انرژی آگاه و چندخدمتی

بارسپاری انرژی کارای مبتنی بر DDQN در MEC. ژو و همکاران مسئله‌ی مشترک تصمیم‌گیری بارسپاری و تخصیص منابع را در سامانه‌ی چندکاربره‌ی پویای MEC بررسی کرده و هدف را کمینه‌کردن مصرف انرژی بلندمدت کل سامانه تحت قید مهلت وظایف قرار داده‌اند [4]. در مدل آن‌ها، وظیفه‌ی کاربر i در اسلات t با سه‌تایی $\Psi_i^t = \{s_i^t, c_i^t, d_{i,\max}^t\}$ توصیف می‌شود؛ که در آن s_i^t حجم داده، c_i^t تعداد سیکل پردازشی، و $d_{i,\max}^t$ حداکثر تأخیر مجاز است. تصمیم‌گیری بارسپاری باینری $x_i^t \in \{0, 1\}$ نشان می‌دهد وظیفه محلی اجرا شود ($x_i^t = 0$) یا به سرور لبه برود ($x_i^t = 1$).

در اجرای محلی، تأخیر و انرژی به‌صورت

$$D_{i,\text{loc}}^t = \frac{c_i^t}{f_i^{\text{loc}}}, \quad E_{i,\text{loc}}^t = c_i^t \delta_i^{\text{loc}}, \quad \delta_i^{\text{loc}} = 10^{-27} (f_i^{\text{loc}})^2 \quad (1-3)$$

تعریف می‌شوند؛ که در آن f_i^{loc} فرکانس پردازنده‌ی کاربر است. در حالت بارسپاری، تأخیر ارسال و اجرای لبه برابر

$$D_{i,\text{off}}^t = \frac{s_i^t}{R_i^t} + \frac{c_i^t}{\beta_i^t f_{\text{MEC}}^t} \quad (2-3)$$

است؛ که R_i^t نرخ، $\beta_i^t \in [0, 1]$ سهم منابع محاسباتی سرور، و f_{MEC}^t ظرفیت کل پردازشی لبه است. انرژی بارسپاری نیز مجموع انرژی پردازش سرور، انرژی ارسال و انرژی بیکاری کاربر هنگام انتظار

است:

$$E_{i,\text{off}}^t = P_{\text{MEC}} \beta_i^t d_{i,\text{exe}}^t + \frac{p_i s_i^t}{R_i^t} + \frac{p_i^w c_i^t}{\beta_i^t f_{\text{MEC}}} \quad (3-3)$$

و انرژی کل کاربر با ترکیب دو حالت به صورت

$$E_i^t = (1 - x_i^t) E_{i,\text{loc}}^t + x_i^t E_{i,\text{off}}^t \quad (4-3)$$

و انرژی سامانه برابر است با

$$E_{\text{total}}^t = \sum_i E_i^t \quad (5-3)$$

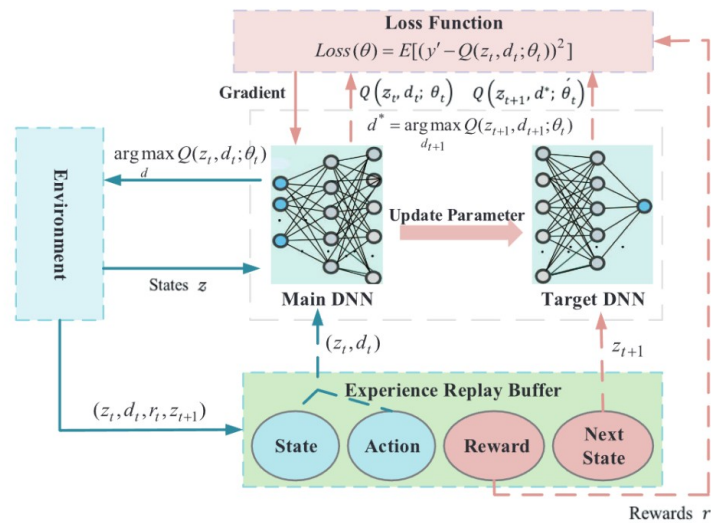
هدف بهینه‌سازی، کمینه کردن میانگین زمانی بلندمدت E_{total}^t تحت قیود باینری بودن تصمیم، رعایت مهلت $(D_i^t \leq d_{i,\text{max}}^t)$ ، و سقف مجموع منابع محاسباتی/طیفی کاربران بارسپاری شده است؛ این مسئله به صورت MINLP و NP-سخت صورت‌بندی می‌شود.

برای حل، مسئله به یک MDP نگاشت می‌شود. حالت شامل انرژی کل و باقیمانده‌ی منابع طیفی/محاسباتی در اسلات جاری است؛ عمل، بردار تصمیم بارسپاری به همراه تخصیص نسبی منابع محاسباتی و طیفی است؛ و پاداش فوری به صورت $r(z_t, d_t) = -E(z_t, d_t)$ تعریف می‌شود تا کمینه کردن انرژی با بیشینه کردن پاداش تجمعی هم‌راستا گردد (در نقض قیود، پاداش تنبیهی اعمال می‌شود). ابتدا روش جدولی Q-یادگیری پیشنهاد شده و سپس برای پرهیز از نفرین ابعاد، نسخه‌ی مبتنی بر DDQN ارائه شده است که تابع ارزش را با شبکه‌ی عصبی تقریب می‌زند. شکل 3-1 معماری عامل DDQN در آن کار را نشان می‌دهد: شبکه‌ی اصلی و هدف، بافر بازپخش تجربه، و به‌روزرسانی پارامترها با گرادینت تابع زیان. نتایج شبیه‌سازی نشان می‌دهد روش DDQN نزدیک به جستجوی کامل عمل می‌کند و نسبت به قواعد پایه (محلی‌اول، بارسپاری‌اول و تصمیم ایستا) انرژی متوسط را کاهش می‌دهد.

این خط پژوهشی با هدف انرژی/بهره‌وری هم‌راستا است و مدل انرژی محلی آن در فصل مفاهیم اولیه‌ی این پایان‌نامه نیز اقتباس شده است؛ با این حال، ارزیابی ژو و همکاران روی مدل انتزاعی MEC چندکاربره است و فاقد اتصال به ترافیک زنده‌ی خودرویی، مدل تأخیر V2X هم‌بسته با صحنه، و سنجش هم‌زمان زمان‌بندی‌پذیری وظایف نرم/سخت در پشته‌ی خودرانی است.

4-3 مصالحه‌ی کیفیت-تأخیر در سناریوهای V2X

در کارهای پیشین، بخشی از پژوهش‌ها مصالحه‌ی کیفیت-تأخیر را در سناریوهای V2X-محور بررسی کرده‌اند [15, 16, 17]. با این حال، در بسیاری از آن‌ها مدل شبکه به صورت انتزاعی یا نیمه‌واقعی در نظر



شکل 3-1: معماری عامل DDQN برای تصمیم مشترک بارسپاری و تخصیص منابع در MEC انرژی کارا: شبکه‌ی اصلی/هدف، بافر بازپخش تجربه و به‌روزرسانی با گرادینان زیان [4].

گرفته می‌شود. در این پایان‌نامه، برخلاف آن رویکردها، تأخیر شبکه به‌عنوان ورودی تصمیم‌گیری و ارزیابی عملیاتی سامانه در حلقه‌ی بسته لحاظ می‌شود (شکل 2-5).

3-5 جمع‌بندی و جایگاه پژوهش حاضر

مرور بالا چهار دسته‌ی اصلی را نشان می‌دهد: (۱) انتخاب سرور و بارسپاری با SARSA/DRL؛ (۲) قابلیت اطمینان از مسیر زمان‌بندی استنتاج عمیق؛ (۳) بارسپاری انرژی آگاه و چندخدمتی؛ (۴) مصالحه‌ی کیفیت-تأخیر در سناریوهای V2X. جدول 3-1 این آثار را از منظر هدف اصلی، روش تصمیم و شکاف نسبت به پژوهش حاضر خلاصه می‌کند.

بر این اساس، شکاف پژوهشی اصلی عبارت است از نبود بستری که هم‌زمان (الف) تصمیم بارسپاری را با یادگیری تحت حالت ترافیک/شبکه بگیرد، (ب) تأخیر واقعی مسیر بارسپاری را در حلقه‌ی ادراک اعمال کند، (ج) وظایف سخت را قابل اطمینان کند، و (د) اثر آن را روی زمان اجرای وظایف نرم، انرژی، و تحمل‌پذیری اشکال وظایف سخت بی‌درنگ بسنجد. فصل‌های بعد این شکاف را با پیاده‌سازی و ارزیابی تجربی پوشش می‌دهند.

جدول 3-1: مقایسه‌ی کارهای پیشین مرتبط با بارسپاری لبه و جایگاه پژوهش حاضر

رویکرد	تمرکز اصلی	روش تصمیم	شکاف نسبت به این پژوهش
انتخاب سرور با DRL [32]	انتخاب سرور MEC	DRL	بدون حلقه‌ی ادراک/کنترل خودرویی
انتخاب سرور با تخمین پارامتر [33]	پایدارسازی تصمیم بارسپاری	تخمین پارامتر + تصمیم	فاقد تأخیر V2X شبکه‌درحلقه
بهینه‌سازی مشترک سرور، بارسپاری و تحویل [34]	شبکه‌ی دسترسی چندگانه	DRL	تمرکز مخابراتی؛ نه ایمنی سخت بی‌درنگ
بارسپاری با SARSA [35]	بارسپاری و تخصیص منابع	SARSA	ارزیابی روی مدل انتزاعی
زمان‌بندی قابلیت اطمینان آگاه DNN [36]	استنتاج بی‌درنگ روی لبه	زمان‌بندی برخط	بدون TMR/مسیر کنترل ایمنی
شتاب‌دهی قابل اطمینان DNN خودرویی [37]	تضمین قابلیت اطمینان در VEC	طراحی سیستمی/زمان‌بندی	بدون سیاست بارسپاری RL آگاه از تراکم
بارسپاری انتخابی انرژی آگاه [38]	استنتاج انتخابی بی‌درنگ	انتخاب بخش‌های استنتاج	بدون هم‌شبه‌سازی ترافیک-شبکه-خودران
بارسپاری انرژی کارای DDQN [4]	کمینه‌کردن انرژی MEC چندکاربره	Q-یادگیری/DDQN	مدل انتزاعی؛ بدون پشته‌ی خودرانی حلقه‌بسته
بارسپاری چندخدمتی فدرال [39]	تخصیص مشترک چند وظیفه	بهینه‌سازی تخصیص	سناریوی غیرخودرانی حلقه‌بسته

فصل 4

پیاده‌سازی

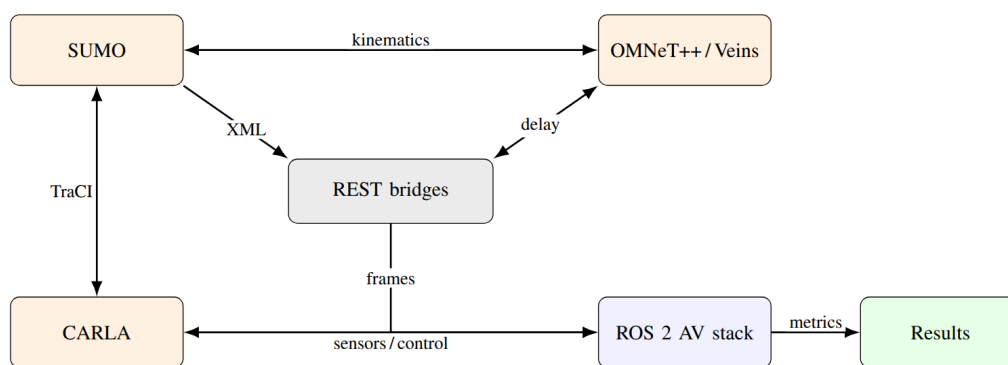
این فصل جزئیات فنی پیاده‌سازی سامانه‌ی پیشنهادی را شرح می‌دهد. تمرکز بر بستر هم‌شبیه‌سازی، پشته‌ی نرم‌افزاری مبتنی بر ROS 2 و چالش‌های عملی پیاده‌سازی است.

1-4 بستر اجرا و ابزارها

سامانه روی میان‌افزار ROS 2 Humble و زبان Python 3.10 پیاده‌سازی شده است. شبیه‌سازی خودروی خودی در CARLA 0.9.15 انجام می‌شود و ترافیک خودروهای غیرخودی از طریق هم‌شبیه‌سازی SUMO تزریق می‌شود. برای مدل‌سازی تأخیر V2X در حالت بارسپاری، از ++OMNeT/Veins استفاده می‌شود. ارتباط بین این لایه‌ها به صورت زیر است:

- ROS 2 میان‌افزار متن‌باز مبتنی بر DDS¹ برای اجرای پیمانه‌ای گره‌ها (node) است؛ داده‌ها عمدتاً از طریق تاپیک (topic) به صورت انتشار-اشتراک مبادله می‌شوند [24]. در این پروژه توزیع Humble به کار رفته است.
- CARLA حسگرها، دینامیک خودرو و فرمان‌های کنترل را تأمین می‌کند و از طریق پل ارتباطی به تاپیک‌های ROS 2 وصل می‌شود؛ با این حال اجرای کامل آن از نظر محاسباتی سنگین است. پل ارتباطی و هسته‌ی اولیه‌ی پشته‌ی خودرانی این پروژه بر پایه‌ی یک مخزن متن‌باز شکل گرفته است [26].

¹Data Distribution Service



شکل 4-1: معماری هم‌شبیه‌سازی: اتصال SUMO، CARLA، Veins/OMNeT++ و پشته‌ی ROS 2.

- SUMO ترافیک اطراف را شبیه‌سازی می‌کند و با CARLA هم‌شبیه‌سازی می‌شود تا موقعیت خودروهای صحنه از SUMO گرفته شود؛ وضعیت زنده‌ی خودروها از طریق TraCI و یک API مبتنی بر HTTP در اختیار تصمیم‌گیرنده و مسیر شبکه قرار می‌گیرد. پارامترهای تولید ترافیک چنان تنظیم شده‌اند که تعداد خودروها در طول آزمایش ثابت و یکنواخت بماند و تزریق تدریجی خودرو در طول زمان انجام نشود؛ هدف، حذف ابهام درباره‌ی تغییر چگالی ترافیک هنگام ارزیابی است. اگرچه CARLA خود دارای NPC‌های دینامیکی است، در این پروژه از NPC‌های سینماتیکی SUMO استفاده می‌شود تا اتصال به OMNeT++ ساده‌تر و بار پردازشی کمتر شود.

- سرور OMNeT موقعیت خودروها و ایستگاه پایه را از SUMO می‌گیرد، درخواست شبیه‌سازی تأخیر را دریافت می‌کند و تأخیر انتقال وظایف بارسپاری شده را برمی‌گرداند.

نمای کلی اتصال ابزارها در شکل 4-1 آمده است.

برای وصل کردن این ابزارها، ترکیبی از پل‌های آماده‌ی متن‌باز و API‌های نوشته‌شده در همین پروژه به کار رفته است. پل ROS 2-CARLA از مخزن متن‌باز موجود گرفته شده و حسگرها و فرمان‌های کنترل را روی تاپیک‌های ROS 2/DDS منتشر می‌کند [26]. هم‌شبیه‌سازی SUMO-CARLA نیز بر پایه‌ی پل هم‌شبیه‌سازی آماده‌ی CARLA است؛ تنها یک لایه‌ی نازک برای تنظیم درگاه TraCI و چندکلاسیتی اضافه شده تا API موقعیت نیز هم‌زمان به SUMO وصل شود. ارتباط داخلی پشته از طریق تاپیک‌های ROS 2 است و دسترسی زنده‌ی موقعیت خودروها به SUMO با پروتکل استاندارد TraCI انجام می‌شود. برای تغذیه‌ی مدل شبکه و مسیر بارسپاری، دو API مبتنی بر HTTP نوشته شد که خلاصه‌ی آن‌ها در جدول 4-1 آمده است.

جدول 4-1: API های نوشته شده برای اتصال شبیه سازها و پروتکل آنها

API	کاربرد	پروتکل
موقعیت SUMO	دریافت وضعیت زندهی خودروها برای مدل شبکه و تصمیم گیرنده	TraCI + (GET) HTTP
تأخیر OMNeT++	اجرای شبیه سازی V2X و بازگرداندن تأخیر بارسپاری	(POST) HTTP

جدول 4-2: بازه ی نمونه برداری یکنواخت پارامترهای وظایف مصنوعی و ادراکی

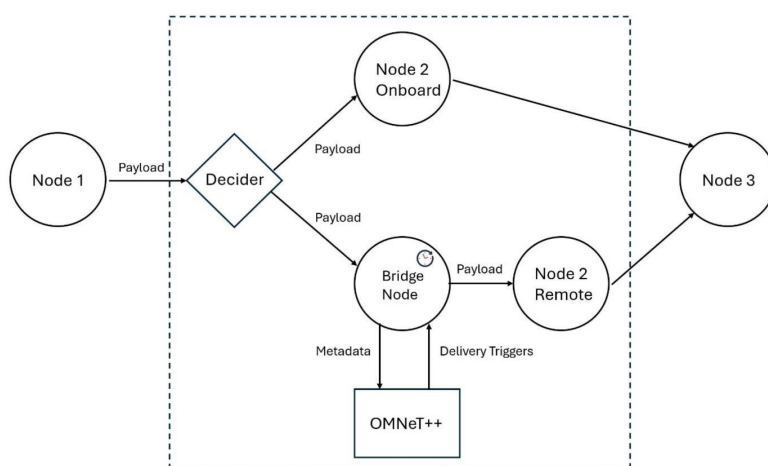
سیکل	اندازه (KB)	وظیفه
$[1/5 \times 10^9, 2 \times 10^9]$	[50, 150]	synthetic 0
$[2 \times 10^9, 3 \times 10^9]$	[100, 300]	synthetic 1
$[3/5 \times 10^9, 4/2 \times 10^9]$	[200, 400]	synthetic 2
$[3/5 \times 10^9, 6 \times 10^9]$	$[100, 1/2 \times 10^5]$	تشخیص چراغ راهنمایی [27]
$[5/5 \times 10^9, 9/5 \times 10^9]$	$[100, 1/2 \times 10^5]$	قطعه بندی معنایی [27]
$[6 \times 10^9, 10/5 \times 10^9]$	$[100, 1/2 \times 10^5]$	تخمین عمق [27]

2-4 پیاده سازی وظایف

وظایف خودرانی معرفی شده در فصل مفاهیم اولیه، بر پایه ی مخزن متن باز ROS 2-CARLA و پل ارتباطی آن به شبیه ساز CARLA پیاده سازی شده اند [26]. برای نقشه برداری و تخمین وضعیت، از بسته ی متن باز LIO-SAM استفاده شده است [30]. علاوه بر وظایف ادراکی و کنترلی فصل دوم، سه وظیفه ی مصنوعی بی درنگ نرم نیز برای ارزیابی بارسپاری افزوده شده اند تا اثر بار محاسباتی افزایشی بر سیاست های محلی/دور دست قابل بررسی باشد. در هر تیک زمانی، برای وظایف مصنوعی و وظایف ادراکی قابل بارسپاری (تشخیص چراغ راهنمایی، قطعه بندی معنایی و تخمین عمق) پارامترهای اندازه و سیکل محاسباتی به صورت یکنواخت تصادفی از بازه ی تعریف شده نمونه برداری می شود؛ بازه های وظایف ادراکی بر اساس محک Chauffeur تنظیم شده اند [27]. بازه های نمونه برداری در جدول 4-2 آمده است.

در جمع بندی، وظایف این پیاده سازی در دو دسته قرار می گیرند:

$$\left\{ \begin{array}{l} \text{تشخیص چراغ راهنمایی (TL)} \\ \text{قطعه بندی معنایی (seg)} \\ \text{تخمین عمق (depth)} \end{array} \right. \quad \text{وظایف نرم؛ پیاده سازی بارسپاری}$$



شکل 4-2: پیاده‌سازی مسیر بارسانی: تصمیم‌گیرنده، اجرای محلی، پل شبکه و نسخه‌ی دوردست.

وظایف سخت؛ پیاده‌سازی قابلیت اطمینان

برنامه‌ریز محلی (طولی PID، عرضی Stanley) کنترلر نقشه‌برداری لیزری (LIO-SAM)

3-4 مسیر بارسانی وظایف

برای شبیه‌سازی بارسانی، گره یک وظیفه دو نسخه شد: یک گره به‌عنوان گره قابل اجرا روی خودرو² و یک گره به‌عنوان گره قابل اجرا روی سرور³. گرهی دوردست با گرهی محلی فرقی ندارد و پیش از اجرای گره دوردست، تنها تأخیر شبکه اعمال می‌شود. شکل 4-2 مسیر پیاده‌سازی تصمیم بارسانی را نشان می‌دهد: پس از ورود داده، تصمیم‌گیرنده بین اجرای محلی و بارسانی انتخاب می‌کند؛ کار گرهی پل⁴ آن است که داده‌های ارسالی را به‌اندازه‌ی تأخیر شبکه به‌دست‌آمده از OMNeT++ در حافظه نگه دارد و پس از آن زمان، داده را به گرهی دوردست بفرستد. در نهایت خروجی هر دو مسیر به مقصد مشترک می‌رسد. برای این کار یک API نوشته شد که به OMNeT++ درخواست می‌زند، موقعیت همه‌ی خودروها را از SUMO می‌فرستد و تأخیر شبکه را دریافت می‌کند.

پس از تصمیم، اجرا به‌صورت زیر است:

- بدون بارسانی: وظایف ادراکی روی گره اصلی اجرا می‌شوند.

²on-board node

³remote node

⁴bridge node

- با بارسپاری: فریم/داده ابتدا به تایپیک میانی منتقل می‌شود؛ گرهی پل تأخیر را از API مربوط به OMNeT++ می‌گیرد، داده را به مدت آن تأخیر در حافظه نگه می‌دارد و سپس به نسخه‌ی دوردست ادراک تحویل می‌دهد. وظایف مصنوعی نیز در همین حلقه زمان‌بندی و ارزیابی می‌شوند تا بار محاسباتی قابل کنترل باشد.

4-4 پیاده‌سازی تحمل‌پذیری اشکال

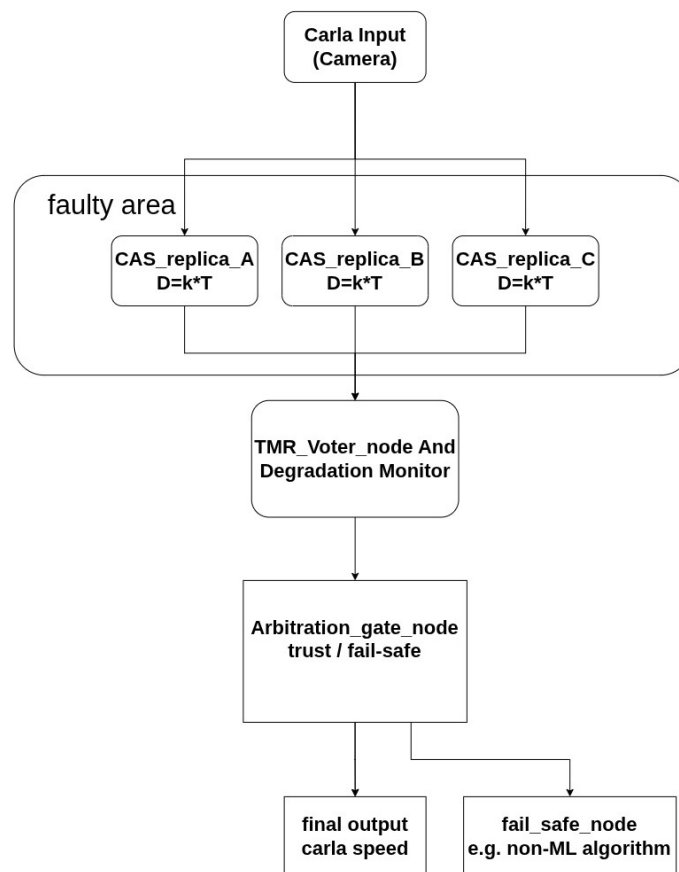
این سازوکار برای کنترلر، سامانه‌ی اجتناب از برخورد (CAS) و LIO-SAM پیاده‌سازی شد. برای وظیفه‌ی سخت بی‌درنگ اجتناب از برخورد، سه نسخه‌ی موازی با دوره‌ی اسمی T و مهلت $D = kT$ اجرا می‌شوند. هر نسخه بردار وضعیت [speed_command, on_time, slack] را منتشر می‌کند. گرهی تزریق خطا می‌تواند روی هر نسخه یکی از حالت‌های سکوت، تأخیر، و ارونگی بیت یا نویز را اعمال کند. رأی‌گیر TMR ابتدا نسخه‌های دیررس/خاموش را حذف می‌کند، سپس با رواداری عددی روی سرعت، اکثریت را انتخاب می‌کند و فرمان نهایی را به محدودکننده‌ی سرعت می‌فرستد. در نبود اکثریت معتبر، سامانه به رفتار ایمن حداقلی تغییر مسیر می‌دهد. شکل 3-4 معماری این مسیر را در ROS 2 نشان می‌دهد: ورودی دوربین CARLA به سه نسخه‌ی موازی CAS می‌رود، رأی‌گیر و پایشگر تنزل، دروازه‌ی داوری اعتماد/fail-safe را تغذیه می‌کند و در نهایت فرمان سرعت نهایی یا مسیر ایمن حداقلی انتخاب می‌شود.

5-4 خط لوله‌ی ارزیابی

پس از راه‌اندازی آزمایش، گرهی معیارها در پنجره‌های زمانی ثابت، تأخیر وظایف، نرخ بارسپاری، کیفیت سرویس و تأخیر شبکه را در قالب JSON ثبت می‌کند. برای ارزیابی قابلیت اطمینان، کمپین تزریق خطا با نرخ‌های مختلف اجرا می‌شود و شاخص‌هایی مانند پوشش ماسک‌شدن خطا و نرخ fail-safe استخراج می‌شود. پس‌پردازش انرژی نیز بر اساس مدل تعریف‌شده در فصل مفاهیم اولیه، انرژی محلی و ارسال را برای هر وظیفه محاسبه می‌کند.

6-4 چالش‌های پیاده‌سازی

یکپارچه‌سازی هم‌زمان شبیه‌سازهای فیزیک، ترافیک، شبکه و پشته‌ی خودرانی، چالش‌های عملی متعددی ایجاد می‌کند که در طراحی بستر لحاظ شده‌اند:



شکل 4-3: پیاده‌سازی تحمل‌پذیر اشکال برای سامانه‌ی اجتناب از برخورد: سه نسخه‌ی CAS، رأی‌گیر TMR، دروازه‌ی داوری و مسیر fail-safe.

- همگام‌سازی چندشبیه‌ساز: گام زمانی CARLA و OMNeT++ با یکدیگر متفاوت است و همگام‌سازی این دو شبیه‌ساز چالش‌برانگیز بود؛ هر ناهماهنگی می‌تواند به تأخیرهای نامعتبر در مسیر بارسپاری منجر شود.
- مسدود شدن صف 2 ROS توسط گره‌ی پل: گره‌ی پل داده‌ی وظیفه‌ی بارسپاری‌شده را تا پایان تأخیر شبکه نگه می‌دارد؛ اگر این نگهداری به‌صورت چندنخی (multi-thread) پیاده‌سازی نشود، ممکن است کل صف پردازش همان وظیفه در ROS 2 متوقف شود و فریم‌های بعدی پشت سر هم معطل بمانند.
- بار محاسباتی هم‌زمان: اجرای ادراک عمیق، تصمیم‌گیرنده‌ی DRL، رأی‌گیر TMR و سه شبیه‌ساز روی یک میزبان، فشار بالایی بر GPU/CPU وارد می‌کند. این سنگینی باعث می‌شود شبیه‌سازی روی هر سیستمی قابل اجرا نباشد. این فشار زمان اجرای محلی را تغییر می‌دهد و در نتیجه خود معیار تأخیر را نیز تحت تأثیر قرار می‌دهد؛ از این رو ثبت معیارها در پنجره‌های ثابت و کنترل بار با

وظایف مصنوعی ضروری است.

- **محیط غیرایستا برای یادگیری:** با تغییر تراکم ترافیک و تأخیر شبکه، دینامیک محیط برای ایجنت بارسپاری غیرایستا است. سیاست اشتراکی و مشاهده‌ی مبتنی بر نواحی، پیچیدگی را کاهش می‌دهد، اما همچنان نیازمند آموزش پایدار در برابر نویز هم‌شبه‌سازی و تغییر سناریو است.
- **محدودیت بی‌درنگی ROS 2: ROS 2** برای توسعه‌ی پیمانه‌ای مناسب است، اما دست را برای تضمین‌های سخت بی‌درنگ چندان باز نمی‌گذارد؛ زمان‌بندی و تأخیر تایپیک‌ها در عمل وابسته به سر بار میان‌افزار و بار میزبان است و کنترل دقیق مهلت‌های سخت را دشوار می‌کند.

7-4 جمع‌بندی فصل

پیاده‌سازی انجام‌شده یک حلقه‌ی بسته‌ی یکپارچه از حسگر تا کنترل فراهم می‌کند که در آن تصمیم بارسپاری، تأخیر شبکه در حلقه و تحمل‌پذیری اشکال هم‌زمان قابل آزمایش‌اند. جزئیات کمی نتایج این بستر در فصل بعد ارائه می‌شود.

فصل 5

ارزیابی و تحلیل نتایج

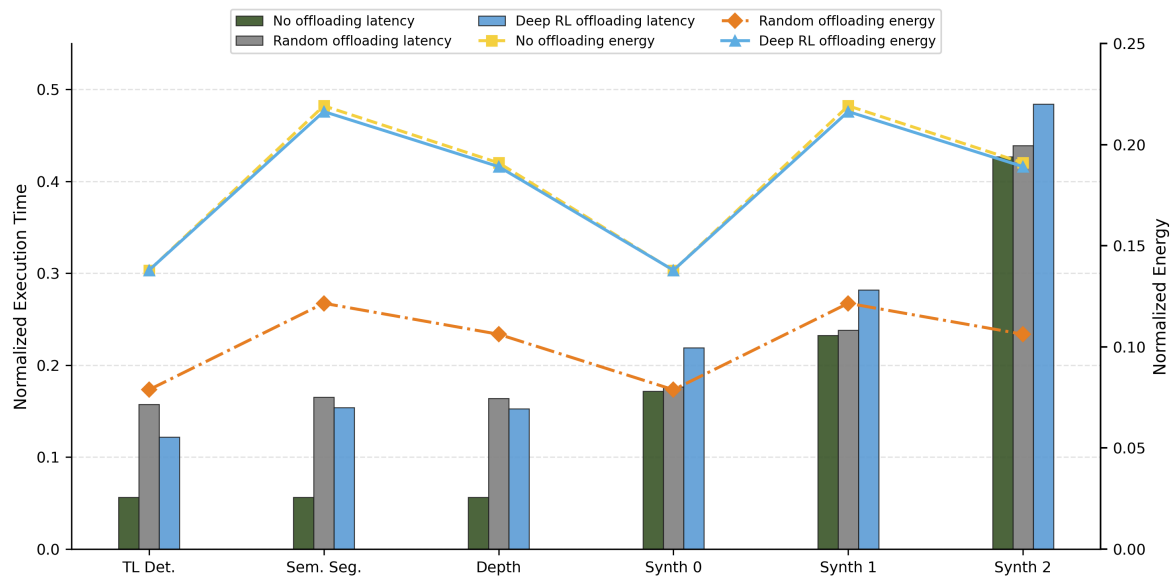
در این فصل نتایج ارزیابی تجربی سیاست‌های بارسپاری و قابلیت اطمینان روی بستر پیاده‌سازی شده ارائه می‌شود. تمرکز بر مقایسه‌ی سه حالت بدون بارسپاری، بارسپاری تصادفی و بارسپاری مبتنی بر یادگیری تقویتی عمیق از نظر زمان اجرای نرمال‌شده و انرژی نرمال‌شده، و نیز نتایج تزریق خطا برای وظایف سخت بی‌درنگ است.

1-5 تنظیم آزمایش

آزمایش‌ها روی پشته‌ی خودرانی حلقه‌بسته اجرا شده‌اند. هر سناریو در شرایط و random seed ثابت، به مدت 500 ثانیه زمان شبیه‌سازی اجرا شده است. تعداد خودروهای ترافیک SUMO در طول هر آزمایش ثابت و یکنواخت نگه داشته می‌شود و تزریق تدریجی خودرو در زمان شبیه‌سازی انجام نمی‌شود. برای هر وظیفه، زمان اجرای انتها به انتها و انرژی نرمال‌شده گزارش می‌شود. زمان اجرا نسبت به مجموع تأخیر حالت بدون بارسپاری نرمال می‌شود و انرژی بر اساس مدل تعریف‌شده در فصل مفاهیم اولیه محاسبه و نرمال می‌شود. وظایف مورد مقایسه عبارتند از تشخیص چراغ راهنمایی، قطعه‌بندی معنایی، تخمین عمق و سه وظیفه‌ی مصنوعی (synthetic 0 تا synthetic 2) با بار محاسباتی افزایشی.

2-5 زمان اجرا و انرژی

شکل 1-5 زمان اجرای نرمال‌شده و انرژی نرمال‌شده را برای سه سیاست نشان می‌دهد.

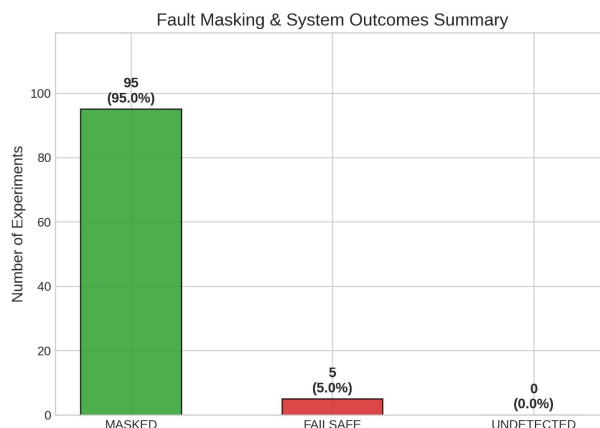


شکل 5-1: مقایسه‌ی زمان اجرای نرمال‌شده و انرژی نرمال‌شده برای سیاست‌های بدون بارسیپاری، تصادفی و یادگیری تقویتی عمیق روی وظایف ادراکی و مصنوعی.

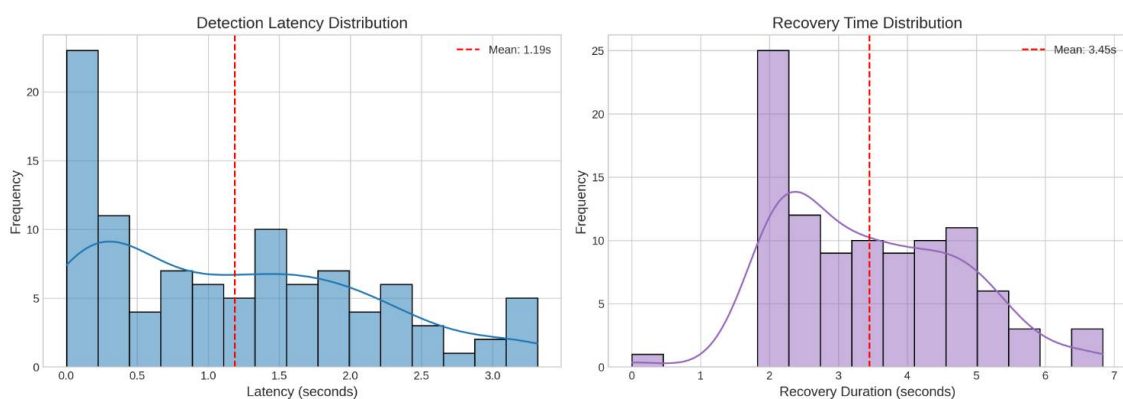
در حالت بدون بارسیپاری، برای وظایف ادراکی سبک‌تر (TL Det.، Sem. Seg. و Depth) کم‌ترین تأخیر انتهابه‌انتها دیده می‌شود؛ با افزایش سنگینی وظایف مصنوعی، زمان اجرا به‌سرعت رشد می‌کند. بین مصرف انرژی کم و زمان اجرای بالا یک مصالحه وجود دارد. برای نمونه، در بارسیپاری تصادفی انرژی نرمال‌شده برای وظایف واقعی پایین است، اما زمان اجرا بالاست؛ در حالت بدون بارسیپاری زمان اجرا برای همین وظایف کم است، ولی مصرف انرژی بالاست چون همه‌ی محاسبات محلی انجام می‌شود. البته در نتایج، برای وظایف مصنوعی در حالت تصادفی زمان اجرا از روش پیشنهادی کم‌تر است؛ دلیلش آن است که زمان شبیه‌سازی OMNeT++ در تأخیر شبکه برای وظایف واقعی به تأخیر شبکه افزوده شده، اما برای وظایف مصنوعی این زمان شبیه‌سازی لحاظ نشده است. همچنین تأخیر شبکه در این آزمایش از سربرار زمانی اجرای مدل کم‌تر است؛ اگر تعداد دستگاه‌های IoT متصل به سرور بیشتر شود، این تأخیر افزایش می‌یابد و انتظار می‌رود مدل پیشنهادی به‌صرفه‌تر باشد.

3-5 قابلیت اطمینان

برای ارزیابی تحمل‌پذیری اشکال، کمپین تزریق خطا با نرخ $\lambda = 0/5$ در 100 آزمایش اجرا شد. شکل 5-2 خلاصه‌ی خروجی سامانه را نشان می‌دهد: در 95٪ موارد خطا با رأی‌گیر TMR ماسک شده، در 5٪ موارد سامانه به مسیر ایمن حداقلی رفته، و هیچ موردی بدون تشخیص باقی نمانده است.



شکل 5-2: خلاصه‌ی ماسک‌شدن خطا و خروجی سامانه در 100 آزمایش تزریق خطا.



شکل 5-3: توزیع تأخیر تشخیص خطا (چپ) و زمان بازیابی (راست).

شکل 5-3 توزیع تأخیر تشخیص خطا و زمان بازیابی را نشان می‌دهد. میانگین تأخیر تشخیص حدود $\frac{1}{19}$ ثانیه و میانگین زمان بازیابی حدود $\frac{3}{45}$ ثانیه است؛ بخش قابل توجهی از تشخیص‌ها در بازه‌ی نزدیک به صفر رخ می‌دهد، در حالی که زمان بازیابی پراکندگی بیشتری دارد.

4-5 جمع‌بندی نتایج

نتایج شکل 5-1 سه پیام کلیدی دارند:

- بدون بارسپاری برای وظایف سبک کم‌تأخیرتر است، اما از نظر انرژی پرهزینه است.
- بارسپاری تصادفی انرژی را کم می‌کند، اما تأخیر را افزایش می‌دهد و می‌تواند مهلت را تهدید کند.

- بار سپاری مبتنی بر یادگیری تقویتی عمیق باید دقیقاً در همین مصالحه قضاوت شود: اثر آن روی زمان اجرا و انرژی در هر وظیفه یکسان نیست.
- در کمپین تزریق خطا، تقریباً همه‌ی موارد یا ماسک شده‌اند یا به مسیر ایمن حداقلی رفته‌اند و مورد تشخیص نشده گزارش نشده است.

فصل 6

نتیجه گیری

این فصل یافته‌های اصلی پایان‌نامه را جمع‌بندی می‌کند، محدودیت‌ها را مرور می‌نماید و مسیرهایی برای کارهای آتی پیشنهاد می‌دهد.

1-6 جمع‌بندی

در این پژوهش یک بستر یکپارچه‌ی دوقلوی دیجیتال برای خودروی خودران طراحی و پیاده‌سازی شد که شبیه‌سازی حسگر/دینامیک (CARLA)، ترافیک اطراف (SUMO)، تأخیر V2X (Veins/OMNeT++) و پشته‌ی نرم‌افزاری مبتنی بر ROS 2 را در یک حلقه‌ی ارزیابی به هم متصل می‌کند. روی این بستر، سه محور اصلی دنبال شد:

- **بارسپاری آگاه از شبکه:** تصمیم اجرای محلی یا دوردست برای وظایف ادراکی و مصنوعی، با خطوط‌پایه‌ی بدون بارسپاری و تصادفی و سیاست یادگیری تقویتی عمیق.
- **ارزیابی چندمعیاره:** سنجش هم‌زمان زمان اجرای محاسبه، تصمیم‌گیرنده و انتقال، و انرژی نرمال شده.
- **تحمل‌پذیری اشکال برای وظایف سخت بی‌درنگ:** پیاده‌سازی سه نسخه‌ی موازی برای کنترلر، اجتناب از برخورد و LIO-SAM، رأی‌گیر TMR و مسیر fail-safe همراه با تزریق خطا.

نتایج تجربی نشان داد بدون بارسپاری معمولاً کم‌تأخیرتر ولی پرمصرف‌تر است؛ بارسپاری تصادفی انرژی را کاهش می‌دهد اما سهم انتقال را بالا می‌برد؛ و سیاست یادگیری تقویتی عمیق باید بر اساس

مصالحه‌ی تأخیر- انرژی هر وظیفه قضاوت شود، نه صرفاً بر اساس یک معیار منفرد. به این ترتیب، بارسپاری انتخابی در خودروهای خودران زمانی مفید است که محیط شلوغ باشد و تأخیر شبکه بالا برود.

2-6 محدودیت‌ها

مهم‌ترین محدودیت‌های کار حاضر عبارتند از:

- مدل انرژی پارامتری است و روی سخت‌افزار واقعی اندازه‌گیری نشده است.
- تأخیر V2X عمدتاً به صورت میانگین روی فریم/وظیفه اعمال می‌شود، نه مدل کاملاً بسته‌به‌بسته¹.
- بخشی از ادراک و تصمیم‌گیری در محیط شبیه‌سازی اعتبارسنجی شده و انتقال مستقیم به داده‌ی واقعی نیازمند بازآموزی/بازطراحی است.
- سیاست یادگیری تقویتی عمیق به کیفیت مشاهده، پاداش و پایداری آموزش وابسته است و در سناریوهای بسیار متفاوت، از جمله تغییر تعداد خودروها، ممکن است تعمیم محدودی داشته باشد.
- وظایف خودرانی به صورت کامل و با بهترین دقت پیاده‌سازی نشده‌اند و بخشی از ورودی‌های ادراکی از برجسب حقیقت زمین (²GT) شبیه‌ساز تأمین می‌شود؛ این کار گراف وابستگی وظایف (³DAG) را قطع می‌کند و از واقع‌گرایی می‌کاهد.

3-6 پیشنهاد کارهای آتی

برای ادامه‌ی پژوهش، مسیرهای زیر پیشنهاد می‌شود:

- اجرای نسخه‌ی کم‌دقت‌تر وظیفه به صورت محلی و هم‌زمان درخواست نسخه‌ی دقیق‌تر و پرهزینه‌تر از سرور، سپس رأی‌گیری روی خروجی‌ها؛ در این طرح وظیفه‌ی سرور بی‌درنگ نرم و وظیفه‌ی محلی بی‌درنگ سخت محسوب می‌شود.
- نوشتن زمان‌بند بی‌درنگ برای ROS 2 و اتصال آن به پشتی خودرانی.

¹packet-by-packet

²Ground Truth

³Directed Acyclic Graph

- اندازه‌گیری انرژی سخت‌افزاری در حلقه⁴ و کالیبراسیون مدل انرژی.
- غنی‌سازی مدل شبکه تا سطح بسته‌به‌بسته و در نظر گرفتن ازدحام متغیر ایستگاه پایه و خارج شدن خودرو از محدوده‌ی پوشش سرور.
- گسترش سیاست بارسازی به حالت پیوسته/چندسطحی و چندسروری به‌جای تصمیم باینری.

⁴hardware-in-the-loop

- [1] P. Li, Z. Xiao, X. Wang, K. Huang, Y. Huang, and H. Gao. EPtask: Deep reinforcement learning based energy-efficient and priority-aware task scheduling for dynamic vehicular edge computing. *IEEE Transactions on Intelligent Vehicles*, 9(1):1830–1846, 2023.
- [2] A. M. Rasouli, M. Esnaashari, and M. Ansari. RASOUL: A reliability-aware task allocation strategy to improve success rate and energy saving in mobile-edge computing. *IEEE Internet of Things Journal*, 14(8):14045–14055, 2025.
- [3] G. M. Hoffmann, C. J. Tomlin, M. Montemerlo, and S. Thrun. Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and tuning. In *2007 American Control Conference*, pages 2296–2301, 2007.
- [4] H. Zhou, K. Jiang, X. Liu, X. Li, and V. C. M. Leung. Deep reinforcement learning for energy-efficient computation offloading in mobile-edge computing. *IEEE Internet of Things Journal*, 9(2):1517–1530, 2021.
- [5] L. Nie, H. Wang, G. Feng, J. Sun, H. Lv, and H. Cui. A deep reinforcement learning assisted task offloading and resource allocation approach towards self-driving object detection. *Journal of Cloud Computing*, 12(1):131, 2023.
- [6] W. Shi, L. Chen, and X. Zhu. Task offloading decision-making algorithm for vehicular edge computing: A deep-reinforcement-learning-based approach. *Sensors*, 23(17):7595, 2023.
- [7] J. A. Ansere, E. Gyamfi, Y. Li, H. Shin, O. A. Dobre, T. Hoang, and T. Q. Duong. Optimal computation resource allocation in energy-efficient edge IoT systems with deep reinforcement learning. *IEEE Transactions on Green Communications and Networking*, 7(4):2130–2142, 2023.

- [8] G. Liu, F. Dai, B. Huang, Z. Qiang, S. Wang, and L. Li. A collaborative computation and dependency-aware task offloading method for vehicular edge computing: A reinforcement learning approach. *Journal of Cloud Computing*, 11(1):68, 2022.
- [9] H. Hussain, M. Zakarya, A. Ali, A. A. Khan, M. R. C. Qazani, M. Al-Bahri, and M. Haleem. Energy efficient real-time tasks scheduling on high-performance edge-computing systems using genetic algorithm. *IEEE Access*, 12:54879–54892, 2024.
- [10] W. Hua, P. Liu, and L. Huang. Energy-efficient resource allocation for heterogeneous edge–cloud computing. *IEEE Internet of Things Journal*, 11(2):2808–2818, 2023.
- [11] Y. Xu, L. Zheng, X. Wu, Y. Tang, W. Liu, and D. Sun. Energy-efficient resource allocation for V2X communications. *IEEE Internet of Things Journal*, 11(18):30014–30026, 2024.
- [12] T. Long, Y. Xia, M. Zhou, J. Li, Y. Ma, and Y. Al-Turki. Fault-tolerant mobile service offloading in mobile edge computing. *IEEE Transactions on Automation Science and Engineering*, 22:17022–17033, 2025.
- [13] Z. Jiang, S. Zhou, X. Guo, and Z. Niu. Task replication for deadline-constrained vehicular cloud computing: Optimal policy, performance analysis, and implications on road traffic. *IEEE Internet of Things Journal*, 5(1):93–107, 2017.
- [14] S. Choi, K. Oh, J. Lee, and B.-S. Kim. Approximate TMR-based six-module high-reliability architecture for sensor data processing. In *2025 IEEE/IEIE International Conference on Consumer Electronics-Asia (ICCE-Asia)*, pages 1–4. IEEE, 2025.
- [15] F. Hawlader, F. Robinet, and R. Frank. Leveraging the edge and cloud for V2X-based real-time object detection in autonomous driving. *Computer Communications*, 213:372–381, 2024.
- [16] D. Jacobellis, M. Ulhaq, F. Racapé, H. Choi, and N. J. Yadwadkar. DeDelayed: Deleting remote inference delay via on-device correction. *arXiv preprint arXiv:2510.13714*, 2025.
- [17] M. Saranya, N. Archana, and G. R. Koushik. Deadline-adherent edge AI for intelligent vehicles: Real-time obstacle and traffic light detection using quantized YOLOv8n on jetson orin nano. *IET Intelligent Transport Systems*, 20(1):e70135, 2026.

- [18] M. Esnaashari, M. Ansari, and A. Ejlali. DReaM: Deep reinforcement learning for joint reliability and power management in multicore IoT devices. *IEEE Internet of Things Journal*, 12(24):54509–54522, 2025.
- [19] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun. CARLA: An open urban driving simulator. In *Conference on Robot Learning*, pages 1–16. PMLR, 2017.
- [20] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner. Microscopic traffic simulation using SUMO. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pages 2575–2582. IEEE, 2018.
- [21] A. Varga and R. Hornig. An overview of the OMNeT++ simulation environment. In *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems & Workshops*, pages 1–10, 2008.
- [22] C. Sommer, R. German, and F. Dressler. Bidirectionally coupled network and road traffic simulation for improved IVC analysis. *IEEE Transactions on Mobile Computing*, 10(1):3–15, 2010.
- [23] M. U. Ahmad, A. Abrar, S. Dasgupta, and M. Rahman. OpenCAMS: An open-source connected and automated mobility co-simulation platform for advancing next-generation intelligent transportation systems research. *arXiv preprint arXiv:2507.09186*, 2025.
- [24] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.
- [25] R. E. Lyons and W. Vanderkulk. The use of triple-modular redundancy to improve computer reliability. *IBM Journal of Research and Development*, 6(2):200–209, 1962.
- [26] asujaykk. GitHub - asujaykk/Self-Driving-Car-CARLA-ROS2: A project to develop a self driving car with carla simulator and ROS2 carla bridge. GitHub, 2026. <https://github.com/asujaykk/Self-Driving-Car-CARLA-ROS2> (accessed July 30, 2026).

- [27] B. Maity, S. Yi, D. Seo, L. Cheng, S.-S. Lim, J.-C. Kim, B. Donyanavard, and N. Dutt. Chauffeur: Benchmark suite for design and end-to-end analysis of self-driving vehicles on embedded systems. *ACM Transactions on Embedded Computing Systems (TECS)*, 20(5s):1–22, 2021.
- [28] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo. SegFormer: Simple and efficient design for semantic segmentation with transformers. *Advances in Neural Information Processing Systems*, 34:12077–12090, 2021.
- [29] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10371–10381, 2024.
- [30] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus. LIO-SAM: Tightly-coupled lidar inertial odometry via smoothing and mapping. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5135–5142. IEEE, 2020.
- [31] X. Liu, J. Zheng, M. Zhang, Y. Li, R. Wang, and Y. He. Multi-user computation offloading and resource allocation algorithm in a vehicular edge network. *Sensors*, 24(7):2205, 2024.
- [32] H. Liu and G. Cao. Deep reinforcement learning-based server selection for mobile edge computing. *IEEE Transactions on Vehicular Technology*, 70(12):13351–13363, 2021.
- [33] H. Xue, D. Zhang, C. Wu, Y. Ji, S. Long, C. Wang, and T. Sato. Parameter estimation-aided edge server selection mechanism for edge task offloading. *IEEE Transactions on Vehicular Technology*, 73(2):2506–2519, 2023.
- [34] T. M. Ho and K.-K. Nguyen. Joint server selection, cooperative offloading and handover in multi-access edge computing wireless network: A deep reinforcement learning approach. *IEEE Transactions on Mobile Computing*, 21(7):2421–2435, 2020.
- [35] T. Alfakih, M. M. Hassan, A. Gumaei, C. Savaglio, and G. Fortino. Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on SARSA. *IEEE Access*, 8:54074–54084, 2020.

- [36] H. Ma, R. Li, X. Zhang, Z. Zhou, and X. Chen. Reliability-aware online scheduling for DNN inference tasks in mobile-edge computing. *IEEE Internet of Things Journal*, 10(13):11453–11464, 2023.
- [37] K. Liu, C. Liu, G. Yan, V. C. S. Lee, and J. Cao. Accelerating DNN inference with reliability guarantee in vehicular edge computing. *IEEE/ACM Transactions on Networking*, 31(6):3238–3253, 2023.
- [38] A. B. Sada, A. Khelloufi, A. Naouri, H. Ning, and S. Dhelim. Energy-aware selective inference task offloading for real-time edge computing applications. *IEEE Access*, 12:72924–72937, 2024.
- [39] P. Tam, S. Math, and S. Kim. Optimized multi-service tasks offloading for federated learning in edge virtualization. *IEEE Transactions on Network Science and Engineering*, 9(6):4363–4378, 2022.

واژه‌نامه

الف

ادراک perception.....
 اودومتری odometry.....
 افزونگی سه‌گانه پیمانه‌ای Triple Modular.....
 Redundancy (TMR)
 انتشار اشتراک publish-subscribe.....
 انتها به انتها end-to-end.....
 انرژی energy.....
 انرژی ارسال transmission energy.....
 اینترنت اشیا Internet of Things (IoT).....
 ایمنی safety.....
 ایمن در برابر خرابی fail-safe.....

پ

پایان به پایان end-to-end.....
 پایشگر monitor.....
 پایشگر برخورد collision monitor.....
 پایشگر تنزل degradation monitor.....
 پشته stack.....
 پشته‌ی خودرانی autonomous driving stack.....
 پل ارتباطی bridge.....
 پیمانه module.....
 پیمانه‌ای modular.....

ت

تأخیر latency.....
 تأخیر شبکه network delay.....
 تابع پاداش reward function.....
 تاپیک topic.....
 تحمل‌پذیری اشکال fault tolerance.....
 تشخیص چراغ راهنمایی traffic light detection...
 تخمین عمق depth estimation.....
 تصمیم‌گیرنده decision maker.....
 تزریق خطا fault injection.....
 تغییر مسیر switch.....
 تنزل degradation.....

ب

بار load.....
 بار سپاری offloading.....
 برنامه‌ریزی planning.....
 برنامه‌ریز محلی local planner.....
 بستن حلقه loop closure.....
 بیشینه‌ی پسین Maximum A Posteriori (MAP)..
 بسته به بسته packet-by-packet.....
 بی‌درنگ real-time.....
 سخت بی‌درنگ hard real-time.....
 نرم بی‌درنگ soft real-time.....

ز	توکار embedded
زمان اجرا execution time.....	
س	چ
سرور لبه edge server.....	چندخدمتی multi-service
سامانه‌ی اجتناب از برخورد Collision Avoidance ...	ح
System (CAS)	حسگری sensing
سامانه‌ی عامل ربات ۲ Robot Operating System 2	خ
(ROS 2)	خط لوله pipeline.....
سرویس توزیع داده Data Distribution Service ...	خودروی خودران autonomous vehicle
(DDS)	خودرانی autonomous driving.....
سخت‌افزاری در حلقه hardware-in-the-loop	خطای سمت heading error
سیاست policy.....	خطای عرضی مسیر cross-track error
ش	د
شبکه‌ی خودرو با همه‌چیز Vehicle-to-Everything ..	دوقلوی دیجیتال digital twin
(V2X)	دوردست remote.....
شبکه‌ی عصبی neural network	ر
شبکه‌ی کیو عمیق دوگانه Double Deep Q-Network	رای‌گیر voter
(DDQN)	رایانش ابری cloud computing.....
شبیه‌سازی simulation.....	رایانش لبه edge computing.....
ص	رایانش لبه‌ی خودرویی Vehicular Edge Computing
صورت‌بندی formulation	(VEC)
ض	رایانش لبه‌ی سیار Mobile Edge Computing.....
ضریب نرم‌سازی softening factor.....	(MEC)
ع	رده class
عملگر actuation.....	رانش drift

ف

فرایند تصمیم مارکوف .. Markov Decision Process (MDP)

فیلتر کالمن .. Kalman filter

فیلتر کالمن توسعه یافته .. Extended Kalman Filter (EKF)

فضای حالت .. state space

ق

قابلیت اطمینان .. reliability

قاب کلیدی .. Keyframe

قطعه بندی معنایی .. semantic segmentation

قید .. constraint

ک

کنترلر .. controller

کنترل طولی .. longitudinal control

کنترل عرضی .. lateral control

کنترلر استتلی .. Stanley controller

کیفیت سرویس .. Quality of Service (QoS)

گ

گره .. node

گره ی پل .. bridge node

گره ی دور دست .. remote node

گره ی محلی .. on-board node

گراف فاکتور .. factor graph

ل

لیدار .. Light Detection and Ranging (LiDAR)

م

محلی .. local

مدل دوچرخه .. bicycle model

محک .. benchmark

مکان یابی و نقشه برداری همزمان .. Simultaneous Localization and Mapping (SLAM)

مصالحه .. trade-off

مهلت زمانی .. deadline

میان افزار .. middleware

ن

نفرین ابعاد .. curse of dimensionality

نرمال سازی .. normalization

نقشه برداری .. mapping

نقشه ی با تفکیک بالا .. High-Definition Map (HD map)

نویز .. noise

و

وارونگی بیت .. bit flip

وظیفه ی مصنوعی .. synthetic task

ه

هم شبیه سازی .. co-simulation

هزینه .. cost

ی

یادگیری تقویتی .. reinforcement learning

یادگیری تقویتی عمیق .. Deep Reinforcement Learning (DRL)

یادگیری ماشین .. machine learning

Abstract

Given the rapid growth of autonomous vehicles and the importance of reducing energy consumption while improving safety in such systems, this project proposes an innovative approach that manages reliability alongside vehicle energy management without compromising safety. Energy-management methods can reduce reliability, and reduced reliability in turn degrades safety in these systems. In recent years, with the growth of Internet-of-Things devices—including intelligent autonomous vehicles—and emerging challenges in cloud computing together with the need for real-time processing, edge computing has emerged as an effective approach to address these challenges. Edge computing is an emerging cloud-computing technology that can deliver better performance than traditional cloud-only processing by reducing latency and improving efficiency. This project jointly improves energy management and reliability in intelligent autonomous vehicles by leveraging machine-learning algorithms and vehicular edge computing. An integrated digital-twin platform was implemented, combining vehicle, traffic, and vehicle-to-everything network simulation with a Robot Operating System 2 autonomous-driving stack, and three policies—no offloading, random offloading, and deep-reinforcement-learning offloading—were evaluated experimentally. The results show that no offloading is typically lower-latency but more energy-intensive, random offloading reduces energy at the cost of higher latency, and the deep-reinforcement-learning policy must be judged by the per-task latency–energy trade-off. In a fault-injection campaign of 100 trials, 95% of faults were masked by the voter, 5% were diverted to the fail-safe path, and no undetected cases were reported.

Keywords: Edge computing, Real-time computing, Intelligent autonomous vehicle, Machine learning, Reliability



Sharif University of Technology
Department of Computer Engineering

B.Sc. Thesis

Machine Learning-Based Reliability and Energy Management in Vehicular Edge Computing

By:

Parsa Shahidi

Supervisor:

Dr. Mohsen Ansari

July 2026